L. คลังข้อสอบซ้อม Socrative ในห้องเรียน (16 ชุด · 376 ข้อ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: คลังคำถาม-เฉลยจากการซ้อม Socrative สด ๆ ในห้องเรียนครบทั้ง 16 ชุด เรียงตามลำดับบท — ทุกข้อมีคำตอบถูกต้อง และข้อคำนวณมีวิธีคิดกำกับในคอลัมน์เฉลย

ความสำคัญกับข้อสอบ: นี่คือ "สไตล์การถามจริง" ของอาจารย์ — แนวถูก/ผิด, ปรนัย A–E และเติมคำตอบ รวมถึงโจทย์โค้ด/กราฟ/ตารางที่ให้รวมก่อนตอบหลายข้อ (แถบข้อมูลสีฟ้า 📊/💻 ต้องอ่านก่อนตอบข้อในช่วงนั้น)

ใช้จริง: อ่านเร็ว ๆ ชุดละ 3 นาที — โฟกัสข้อคำนวณ (Scheduling · Paging · Segmentation · Page Fault · inode/Disk) ที่อาจารย์ออกซ้ำทุกปี และข้อ True/False ที่เป็นกับดักคำต่อคำ

L1. Introduction to OS — ชุดที่ 1 · 40 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ระบบปฏิบัติการคือฮาร์ดแวร์ตัวหนึ่งในระบบคอมพิวเตอร์	**ไม่จริง**	False (ระบบปฏิบัติการเป็นซอฟต์แวร์ระบบ ไม่ใช่ฮาร์ดแวร์)
2	ระบบปฏิบัติการจะควบคุมฮาร์ดแวร์เท่านั้น	**ไม่จริง**	False (OS ควบคุมและจัดการทั้งฮาร์ดแวร์และซอฟต์แวร์/โปรแกรมประยุกต์)
3	หน้าที่หนึ่งของระบบปฏิบัติการคือการทำให้โปรแกรมทำงานร่วมกันได้	**จริง**	True
4	หน้าที่หนึ่งของระบบปฏิบัติการคือการให้ผู้ใช้ใช้งานคอมพิวเตอร์ตามสิทธิที่ได้รับ	**จริง**	True
5	ในวิชานี้หน่วยประมวลผลจะรวมถึง GPU ด้วย	**ไม่จริง**	False (ในบริบทพื้นฐานของวิชานี้ หน่วยประมวลผลหลักหมายถึง CPU)
6	ในวิชานี้หน่วยความจำจะหมายถึง RAM เท่านั้น	**จริง**	True (ในเนื้อหาวิชานี้ หน่วยความจำหลักหรือ Main Memory อ้างถึง RAM เท่านั้น)
7	การติดตามการใช้งานทรัพยากรเป็นหน้าที่ที่ต้องมีในแต่ละองค์ประกอบของระบบปฏิบัติการ	**จริง**	True
8	องค์ประกอบบางตัวของระบบปฏิบัติการไม่จำเป็นต้องมีการควบคุมการเข้าถึงทรัพยากรที่มันดูแล	**ไม่จริง**	False (ทุกองค์ประกอบที่ดูแลทรัพยากรจำเป็นต้องมีการควบคุมการเข้าถึงเสมอ)
9	ส่วนของระบบปฏิบัติการที่ทำหน้าที่ที่สำคัญในการจัดการระบบคอมพิวเตอร์คือ	**D** kernel	D. kernel
10	โปรแกรมที่ใช้โหลดเคอร์เนลขึ้นมาทำงานมีชื่อเรียกรวม ๆ ว่า	**C** bootloader	C. bootloader
11	ชื่อของโปรแกรม boot loader ที่เป็นที่นิยมใช้บนระบบปฏิบัติการ Linux	**E** GRUB	E. GRUB
12	โปรแกรมที่ใช้โหลด boot loader ถ้าใช้ BIOS โปรแกรมดังกล่าวจะถูกเก็บอยู่ใน	**B** MBR	B. MBR (Master Boot Record)
13	โปรแกรมที่ใช้โหลด boot loader ถ้าใช้ UEFI โปรแกรมดังกล่าวจะถูกเก็บอยู่ใน	**E** ESP	E. ESP (EFI System Partition)
14	เมื่อเราเปิดเครื่องคอมพิวเตอร์โปรแกรมแรกที่จะถูกโหลดเข้ามารันคือ	**D** UEFI	D. UEFI (หรือ BIOS ในเครื่องรุ่นเดิม)
15	โปรแกรม BIOS มักจะถูกเก็บไว้ใน ของเครื่องคอมพิวเตอร์	**B** ROM	B. ROM
16	ถ้าใช้ UEFI การแสดงรายการของ OS ที่ติดตั้งอยู่ในเครื่องสามารถทำได้โดยไม่ต้องไปอ่านข้อมูลจาก MBR	**จริง**	True (UEFI บันทึก Boot Entries ไว้ใน NVRAM/ESP)
17	ข้อดีประการเดียวที่ BIOS เหนือกว่า UEFI คือมันทำงานเร็วกว่า	**ไม่จริง**	False (UEFI ทำงานเร็วกว่าและรองรับคุณสมบัติสมัยใหม่มากกว่า)
18	จะต้องมีการกันเนื้อที่หน่วยความจำส่วนหนึ่งไว้ให้ระบบปฏิบัติการใช้	**จริง**	True (สงวนไว้สำหรับ OS Kernel และตารางข้อมูลของระบบ)
19	ใน Dual Mode Operation คำสั่งที่จะถูกเรียกใช้เมื่อเครื่องอยู่ใน kernel mode เท่านั้นเรียกว่า	**D** Privileged Instruction	D. Privileged Instruction
20	คำสั่งอ่านข้อมูลวันที่และเวลาปัจจุบันของระบบไม่จำเป็นต้องเป็น Privileged Instruction	**จริง**	True (การอ่านค่าไม่อันตรายต่อความมั่นคงของระบบ จึงไม่ต้องเป็น privileged instruction)
21	การที่เราติดตั้งระบบปฏิบัติการมากกว่าหนึ่งตัวลงในคนละ partition ของฮาร์ดดิสก์เรียกว่าการทำ virtual machine	**ไม่จริง**	False (เรียกว่า Multi-boot หรือ Dual-boot)
22	คำสั่งกำหนดค่าให้กับตัวแปรควรเป็น privileged instruction เพราะมีการใช้งานหน่วยความจำซึ่งถ้าไม่ป้องกันอาจมีโปรแกรมไม่ประสงค์ดี ใช้งานหน่วยความจำได้อย่างอิสระ	**ไม่จริง**	False (การกำหนดค่าตัวแปรใน User Space เป็นคำสั่งพื้นฐานของโปรแกรมทั่วไป ไม่ใช่ privileged instruction)
23	ถ้าเรารันโปรแกรม Oracle Virtual Box ภายใต้ระบบปฏิบัติการ Mac OS และใช้โปรแกรม Oracle Virtual Box สร้าง Virtual Machine และติดตั้งระบบปฏิบัติการ Windows 10 ลงบน virtual machine ดังกล่าว โปรแกรม Oracle Virtual Box เป็นโปรแกรมที่เรียกว่า	**A** Virtualization Software	A. Virtualization Software (หรือ Hypervisor Type 2)
24	จากสถานการณ์ข้อ 23 Mac OS เป็นโปรแกรมที่เรียกว่า	**C** Host OS	C. Host OS
25	จากสถานการณ์ข้อ 23 Windows 10 เป็นโปรแกรมที่เรียกว่า	**B** Guest OS	B. Guest OS
26	ถ้าเรามี Windows 10 เป็น Host OS เราไม่สามารถสร้าง virtual Machine ที่ติดตั้ง Windows 10 ได้ เพราะมันจะทำงานขัดแย้งกัน	**ไม่จริง**	False (สามารถสร้างและรัน OS เดียวกันซ้อนกันได้เนื่องจากแยกสิ่งแวดล้อมกันอย่างสมบูรณ์)
27	แนวคิดของ Virtual Machine สามารถนำไปใช้กับระบบ Cloud ได้ด้วย	**จริง**	True
28	แนวคิดของ container technology คือการรวมเอา application และ guest OS เข้าไว้ด้วยกัน	**ไม่จริง**	False (Container แชร์ Host OS kernel ร่วมกัน ไม่ได้มี Guest OS เต็มตัว)
29	แนวคิดของ Virtual Machine สามารถนำไปใช้กับภาษาโปรแกรมได้ด้วย	**จริง**	True (เช่น Java Virtual Machine: JVM)

30	container แต่ละตัว จะทำงานบนสภาพแวดล้อมของตัวเอง ไม่ขึ้นกับสภาพแวดล้อมของ container ตัวอื่น	**จริง**	True
31	โปรแกรมหลักในการจัดการการทำงาน Container Technology ของ Docker เรียกว่า	**A** docker engine	A. docker engine
32	สิ่งที่เรา download จาก docker hub คือ	**B** docker image	B. docker image
33	สิ่งที่เรารันจากสิ่งที่เรา download จาก docker hub คือ	**C** docker container	C. docker container
34	หนึ่ง docker image สามารถรันได้แค่หนึ่ง docker container	**ไม่จริง**	False (Image เดียวสร้าง Container ได้ไม่จำกัด)
35	container ที่รันอยู่บนเครื่องเดียวกันจะใช้ Host OS ตัวเดียวกัน	**จริง**	True
36	คำสั่ง docker ที่ใช้รัน container จาก docker image คือ	**A** docker run	A. docker run
37	คำสั่ง docker ที่ใช้แสดง docker image บนเครื่องเราคือ	**B** docker images	B. docker images
38	คำสั่ง docker ที่ใช้เรียกให้ container ที่เคยรันอยู่แล้ว แต่เราสั่งหยุดไป ให้กลับมาทำงานคือ	**C** docker start	C. docker start
39	คำสั่ง docker ที่ใช้แสดง container ที่อยู่บนเครื่องเราคือ	**E** docker ps	E. docker ps
40	คำสั่ง docker rm คือการลบ docker image ออกจากเครื่องของเรา	**ไม่จริง**	False (`docker rm` ใช้ลบ container, ถ้าลบ image ต้องใช้ `docker rmi`)

L2. CompOr and OS & Categories of OS — ชุดที่ 2.1 · 16 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	Device controller จะบอกว่ามันทำงานเสร็จแล้วโดยใช้	**B** Interrupt	B. Interrupt
2	โปรแกรมที่ใช้จัดการ interrupt เรียกว่า	**C** Interrupt Service Routine	C. Interrupt Service Routine (ISR)
3	การค้นหาโปรแกรมจัดการ interrupt จะทำผ่าน	**B** Interrupt vector	B. Interrupt vector (ตารางเก็บแอดเดรสของ ISR)
4	ระบบที่เมื่อทำงานแล้วจะทำงานไปจนจบโดยไม่มีการติดต่อกับผู้ใช้คือ	**A** Batch System	A. Batch System
5	คุณสมบัติที่สำคัญของระบบปฏิบัติการแบบโต้ตอบคือการแบ่งเวลาการใช้งาน CPU	**จริง**	True (Time-sharing)
6	ในระบบ hybrid งานแบบ batch จะทำงานหลังฉาก	**จริง**	True (Background process)
7	ความสามารถที่ OS ทั้งแบบที่เป็น Multi-Program Batch และ Interactive ต้องมีร่วมกันก็คือ การเลือกโปรแกรมที่อยู่ในหน่วยความจำเข้ามาใช้งาน CPU	**จริง**	True (CPU Scheduling)
8	ในวิชานี้ Windows 11 เป็นตัวอย่างของระบบปฏิบัติการแบบ	**B** Single-user multi-tasking	B. Single-user multi-tasking
9	ในวิชานี้ การที่ OS ยอมให้สร้างบัญชีผู้ใช้ได้หลาย ๆ บัญชี จะถือว่ามันเป็น multi-user OS	**ไม่จริง**	False (Multi-user ต้องยอมให้ผู้ใช้หลายคนล็อกอินใช้งานระบบและทรัพยากรพร้อมกันได้จริง เช่น UNIX/Linux)
10	ระบบซึ่งต้องตอบสนองในเวลาที่กำหนด	**C** Real Time System	C. Real Time System
11	ระบบ Real Time ต้องการ OS ที่พัฒนาขึ้นมาโดยเฉพาะเท่านั้น	**ไม่จริง**	False (สามารถดัดแปลง General OS หรือใช้ RTOS ก็ได้)
12	ระบบที่มี CPU หลายตัวในเครื่องเดียวเรียกว่า	**E** Parallel System	E. Parallel System (Tightly-coupled system)
13	ระบบปฏิบัติการที่เราใช้ในปัจจุบันส่วนใหญ่รองรับ parallel system	**จริง**	True (รองรับ Multi-core processor)
14	Loosely coupled system	**D** Distributed System	D. Distributed System
15	ถ้าใช้ระบบปฏิบัติการแบบกระจาย (distributed OS) ทรัพยากรจะถูกควบคุมโดย OS ของแต่ละโหนดในเครือข่าย	**จริง**	True
16	ในวิชานี้ Windows 11 จัดเป็น Network OS	**จริง**	True (Windows ยุคปัจจุบันมีคุณสมบัติรองรับการทำงานด้านเน็ตเวิร์ก แชร์ไฟล์ แชร์เครื่องพิมพ์ในตัว)

L3. OS Structure — ชุดที่ 2.2 · 16 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ส่วนของ OS ที่จัดเตรียมส่วนติดต่อให้โปรแกรมที่เขียนขึ้นเรียกใช้เพื่อใช้บริการของ OS	**B** System Calls	B. System Calls
2	บริการของ OS ที่เกี่ยวกับการวางกฎเกณฑ์ และควบคุมให้เป็นไปตามกฎเกณฑ์ในการใช้งานทรัพยากรของระบบ	**C** Protection	C. Protection
3	บริการของ OS ที่เกี่ยวกับการตรวจสอบสิทธิการเข้าใช้งานระบบ	**D** Security	D. Security (เช่น การยืนยันตัวตนและการเข้าถึงจากภายนอก)
4	บริการด้านการสื่อสารระหว่างโปรแกรมที่ OS เตรียมไว้ให้	**C** message passing	C. message passing (IPC แบบหนึ่ง)
5	Command Line Interface (CLI) ไม่สามารถอยู่ใน kernel ได้	**ไม่จริง**	False (ในบางสถาปัตยกรรม CLI สามารถฝังอยู่ในส่วนของ kernel ได้)
6	`ls` เป็นตัวอย่างคำสั่งแบบ command built-in ของ Linux	**ไม่จริง**	False (`ls` เป็น external program อยู่ที่ `/bin/ls`)
7	`cd` เป็นตัวอย่างคำสั่งแบบ command built-in ของ Linux	**จริง**	True (`cd` ถูกฝังอยู่ใน shell โดยตรง)
8	คำสั่งที่เป็น command built-in จะเริ่มต้นทำงานได้เร็วกว่าคำสั่งที่เป็นโปรแกรมภายนอก	**จริง**	True (ไม่ต้องโหลดไฟล์ binary จาก disk เข้า memory)
9	คำสั่งที่เป็น command built-in จะแก้ไขได้ง่ายกว่าคำสั่งที่เป็นโปรแกรมภายนอก	**ไม่จริง**	False (คำสั่ง built-in ต้องแก้ที่ตัวเชลล์แล้วคอมไพล์ใหม่)
10	OS จัดการการเรียกใช้ system call คล้ายกับการจัดการ hardware interrupt	**จริง**	True (ใช้ Trap/Software Interrupt ผ่าน Vector Table เหมือนกัน)
11	การใช้ system call เราจะต้องเขียนโปรแกรมเรียกใช้ฟังก์ชันของ System Call (ไม่ว่าจะเป็นระดับ OS หรือ high level API) โดยตรงเท่านั้น	**ไม่จริง**	False (สามารถเรียกผ่าน Library ทั่วไป เช่น Standard C Library `printf()`, `fopen()` ซึ่งจะไปเรียก system call ต่ออีกทอดหนึ่ง)
12	การใช้ high-level API มีข้อดีคือโปรแกรมจะทำงานเร็วกว่าเรียกใช้ System call ระดับ OS	**ไม่จริง**	False (API มี overhead มากกว่า จึงไม่เร็วกว่าการเรียก native system call โดยตรง)
13	การจัดการ System Call จะเหมือนกับการจัดการ Interrupt	**จริง**	True
14	การเรียกใช้ System Call ก็คือการ interrupt ทางซอฟต์แวร์	**จริง**	True (เรียกว่า Software Interrupt หรือ Trap)
15	สถาปัตยกรรมของ kernel ที่ให้ประสิทธิภาพสูงสุดคือ	**B** Monolithic	B. Monolithic (บริการทุกอย่างอยู่ใน Address Space เดียวกัน ทำให้เรียกใช้งานได้เร็ว ไม่ต้องสลับ context บ่อย)
16	สถาปัตยกรรมของ kernel ที่น่าจะทำให้การปรับปรุง OS ทำได้ง่ายที่สุดคือ	**A** Microkernel	A. Microkernel (บริการส่วนใหญ่ถูกแยกเป็น User Service ทำให้แก้ไขเพิ่มเติมได้โดยไม่ต้องรื้อ Kernel)

L4. Introduction to Process — ชุดที่ 3 · 24 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	จากนิยามของ OS ไฟล์ `paint.exe` คือ process	**ไม่จริง**	False (ไฟล์บนดิสก์คือ Program ส่วน Process คือโปรแกรมที่ถูกโหลดขึ้นมาทำงานในหน่วยความจำแล้ว)
2	การสร้าง process จากโปรแกรมจะเป็นการจับคู่แบบหนึ่งต่อหนึ่ง คือ หนึ่งโปรแกรมจะสร้างได้หนึ่ง process	**ไม่จริง**	False (หนึ่งโปรแกรมสามารถถูกเปิดใช้งานพร้อมกันได้หลาย process)
3	แต่ละ process จะมี data section ของตัวเอง	**จริง**	True
4	โพรเซสที่รอให้ OS มาเลือกมันไปใช้งาน CPU จะอยู่ในสถานะ	**B** ready	B. ready
5	สมมติโพรเซสอยู่ในสถานะ running ถ้ามันต้องการทำ I/O มันจะเปลี่ยนสถานะเป็น	**D** waiting	D. waiting
6	การเปลี่ยนสถานะของโพรเซสในข้อใดต่อไปนี้ไม่สามารถเกิดขึ้นได้	**D** waiting → running	D. waiting → running (เมื่อทำ I/O เสร็จ ต้องกลับไปต่อคิวที่ ready ก่อนเสมอ จะกระโดดไป running ทันทีไม่ได้)
7	ข้อมูลของโพรเซสแต่ละตัวจะถูกเก็บไว้ใน	**D** PCB	D. PCB (Process Control Block)
8	โพรเซสที่รอการใช้งานเครื่องพิมพ์จะอยู่ใน	**C** Device queue	C. Device queue
9	กระบวนการสลับโพรเซสเข้าใช้งาน CPU เรียกว่า	**C** Context Switch	C. Context Switch
10	การทำ context switch จะเกิดจากส่วนของ OS ที่เรียกว่า	**D** Dispatcher	D. Dispatcher (Dispatcher เป็นตัวสลับ Context จริง ส่วน Scheduler เป็นตัวเลือกว่าจะเอาโปรเซสไหน)
11	ส่วนของ OS ที่ใช้เลือกโพรเซสที่อยู่ใน ready queue เพื่อให้เข้าไปใช้งาน CPU คือ	**E** Scheduler	E. Scheduler
12	แต่ละ process จะสร้าง process ย่อยได้ไม่เกินสาม process	**ไม่จริง**	False (สร้างได้ตามที่ทรัพยากรระบบจะอำนวย)
13	สมมติใช้คำสั่งต่อไปนี้ในการคอมไพล์ source code ภาษา C ชื่อ main.c `gcc main.c -o main.exe` สมมติในโปรแกรมไม่มี error ใด ๆ จะได้โปรแกรมชื่อ	**C** main.exe	C. main.exe (แฟล็ก `-o` ระบุชื่อไฟล์ output)
14	หลังจากการเรียกใช้ ฟังก์ชัน `fork()` child process จะเริ่มทำงานจากตำแหน่งเริ่มต้นของ program (เช่น เริ่มทำงานตามคำสั่งแรกที่เขียนไว้ใน main program)	**ไม่จริง**	False (Child process จะเริ่มทำงานต่อจากคำสั่ง `fork()` ทันที ไม่ได้กลับไปเริ่มที่ต้นฟังก์ชัน `main`)
15	หลังจากการเรียกใช้ ฟังก์ชัน `fork()` การเปลี่ยนค่าตัวแปรของ child process จะไม่ทำให้ตัวแปรชื่อเดียวกันใน parent process เปลี่ยนค่าตามไปด้วย	**จริง**	True (ทั้งสองมี Address Space แยกจากกัน)
16	ในโค้ดโปรแกรมเราสามารถแยกส่วนของโค้ดที่ต้องการให้ parent process ทำงาน กับส่วนที่ต้องการให้ child process ทำงาน โดยดูจากค่าที่ฟังก์ชัน `fork()` ส่งกลับมา	**จริง**	True (Child ได้รับค่า 0, Parent ได้รับค่า PID ของลูก)
17	OS จะให้สิทธิ parent process สูงกว่า child process คือหลังจาก fork OS จะให้ parent process ทำงานก่อน child process	**ไม่จริง**	False (ลำดับการทำงานขึ้นอยู่กับ CPU Scheduler ตัวใดอาจทำงานก่อนก็ได้)
18	ฟังก์ชัน `fork()` จะ return ค่าลบให้กับ child process ในกรณีที่ `fork()` ทำงานผิดพลาด (สร้าง process ไม่ได้)	**ไม่จริง**	False (ถ้าเกิดข้อผิดพลาด จะไม่มี child process ถูกสร้างขึ้น และจะ return ค่าลบให้กับ parent process เท่านั้น)
19	ผลลัพธ์ที่ได้จากการทำงานของโปรแกรมที่มีการสร้าง process ย่อย อาจมีได้มากกว่าหนึ่งแบบ	**จริง**	True (ขึ้นอยู่กับการสลับคิวของ Scheduler)

20	```
	/* fork_example1.c */
	#include <stdio.h>
	#include <stdlib.h>
	#include <unistd.h>
	#include <sys/types.h>
	#include <sys/wait.h>
	
	int main() {
	    pid_t pid;
	    int i;
	    for (i = 0; i < 2; i++) {
	        pid = fork();
	        if (pid == 0) {
	            printf("child %d\n", i + 1);
	            exit(0);
	        }
	    }
	    while(wait(NULL) != -1);
	    printf("goodbye\n");
	    return 0;
	}
	``
	จากโปรแกรมดังรูป จะมีโพรเซสถูกสร้างขึ้นกี่โพรเซส (รวมทั้ง parent process ด้วย)
		C 3	C. 3 รอบ i=0 แม่สร้างลูก 1 ซึ่งลูกปริ้นท์แล้ว `exit(0)` ทันที, รอบ i=1 แม่สร้างลูก 2 ซึ่งลูกปริ้นท์แล้ว `exit(0)` เช่นกัน รวม Parent (1) + Child1 (1) + Child2 (1) = 3 โพรเซส

21	จากโปรแกรมในข้อ 20 ผลลัพธ์ที่เป็นไปได้คือ	**A.** - B. - C. - D. ``` child 1 goodbye child 2 ```	A (เนื่องจากมีคำสั่ง `while(wait(NULL) != -1);` ทำให้ Parent ต้องรอลูกทุกตัวทำงานเสร็จก่อน คำว่า "goodbye" จึงต้องอยู่บรรทัดสุดท้ายเสมอ)

22	```
	#include <stdio.h>
	#include <stdlib.h>
	#include <unistd.h>
	#include <sys/types.h>
	#include <sys/wait.h>
	
	int main() {
	    pid_t pid;
	    int i;
	    for (i = 0; i < 2; i++) {
	        pid = fork();
	        if (pid == 0) {
	            printf("child %d\n", i + 1);
	        }
	    }
	    while(wait(NULL) != -1);
	    printf("goodbye\n");
	    return 0;
	}
	``
	จากโปรแกรมดังรูป (ไม่มีคำสั่ง `exit(0)` ใน child) จะมีโพรเซสถูกสร้างขึ้นกี่โพรเซส (รวมทั้ง parent process ด้วย)
		D 4	D. 4 ($2^2 = 4$ โพรเซส เพราะลูกตัวแรกไม่ได้ exit จึงวนลูปไป fork ลูกตัวที่สองต่อด้วย)

23	จากโปรแกรมในข้อ 22 คำว่า "goodbye" จะถูกพิมพ์ออกมากี่ครั้ง	**D** 4	D. 4 (ทั้ง 4 โพรเซสหลุดออกจากลูปแล้วรันมาถึงบรรทัด `printf("goodbye\n");`)
24	จากโปรแกรมในข้อ 22 คำว่า "child 2" จะถูกพิมพ์ออกมากี่ครั้ง	**B** 2	B. 2 (ในลูป i=1 จะมีโพรเซสแม่ และลูกตัวแรก ทำการ fork ทำให้เกิดลูกใหม่ 2 ตัวที่มี `pid == 0` ในรอบ i=1 จึงพิมพ์ "child 2" ทั้งหมด 2 ครั้ง)

L5. Fork and Exec — ชุดที่ 4.1 · 19 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	System call ที่ใช้ในการเรียกโปรแกรมอื่นขึ้นมาทำงานคือ	**A** execl	A. execl (กลุ่มฟังก์ชัน exec)
2	System call ที่ใช้ในการเรียกโปรแกรมอื่นขึ้นมาทำงาน ที่เราต้องระบุพารามิเตอร์ในรูปของอาเรย์คือ	**B** execv	B. execv (`v` หมายถึง Vector/Array)
3	System call ที่ Parent Process ใช้เพื่อรอ Child Process จบการทำงาน	**E** wait	E. wait
4	System call ที่ Process ใช้จบการทำงานและส่งค่ากลับให้ Parent Process	**C** exit	C. exit
5	คำสั่งบนระบบปฏิบัติการ UNIX ที่ใช้แสดงสถานะของโพรเซส	**A** ps	A. ps
6	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execlp("/bin/ls", "-l", NULL);` ผลลัพธ์ที่ได้ก็เหมือนกับการป้อนคำสั่ง ls -l	**ไม่จริง**	False (เนื่องจากการเรียก `execlp` พารามิเตอร์ตัวแรกใน list ควรเป็นชื่อโปรแกรม หรือ `argv[0]` ในที่นี้ส่ง `"-l"` ไปเป็น `argv[0]` ทำให้คำสั่งทำงานผิดรูปแบบ ไม่เทียบเท่ากับการสั่ง `ls -l`)
7	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execlp("/bin/ls", "ly", "-l", NULL);` จะทำให้การเรียกใช้ฟังก์ชัน execlp ทำงานไม่สำเร็จ นั่นคือจะไม่มีการเรียกใช้คำสั่ง ls	**ไม่จริง**	False (คำสั่ง `ls` จะยังคงถูกโหลดมาทำงานสำเร็จ เพียงแต่ค่า `argv[0]` จะมีค่าเป็นคำว่า `"ly"`)
8	ถ้าในโค้ดมีการเขียนคำสั่งนี้ `execl("ls", "ls", "-l", NULL);` ถ้าคำสั่ง ls ไม่อยู่ในโฟลเดอร์เดียวกับโปรแกรมที่มีคำสั่งนี้ คำสั่ง execl จะทำงานไม่สำเร็จ นั่นคือจะไม่มีการเรียกใช้คำสั่ง ls	**จริง**	True (ฟังก์ชัน `execl` ธรรมดา ไม่ค้นหาจากตัวแปรแวดล้อม `PATH` เหมือน `execlp` ดังนั้นถ้าไม่ระบุ full path และไฟล์ไม่อยู่ใน directory ปัจจุบันจะรันไม่สำเร็จ)
9	ถ้าต้องการเขียนโปรแกรมเพื่อเรียกใช้คำสั่ง cd เพื่อเปลี่ยนไปยังโฟลเดอร์ test ที่อยู่ภายใต้โฟลเดอร์ปัจจุบันที่โปรแกรมทำงานอยู่ เราจะเขียนคำสั่งดังนี้ `execlp("cd", "cd", "./test", NULL);`	**ไม่จริง**	False (คำสั่ง `cd` เป็น Shell built-in ไม่มีไฟล์ executable แยกต่างหากใน disk ให้เรียกผ่าน `execlp` ได้)

10	สมมติเขียน shell script ดังนี้
	`mkdir test`
	`cd test`
	จากนั้นจัดเก็บไว้ในไฟล์ชื่อที่เพิ่งสร้างขึ้น ถ้าเรียกใช้ shell script นี้ โดยใช้คำสั่ง
	`bash test.sh`
	สมมติโฟลเดอร์ test ยังไม่มีอยู่ในโฟลเดอร์ปัจจุบันที่เราเรียกใช้งาน shell script นี้ เมื่อโปรแกรมทำงานจบ โฟลเดอร์ปัจจุบันที่เราจะใช้งานอยู่คือโฟลเดอร์ test ที่เพิ่งสร้างขึ้น
		ไม่จริง	False (การรันด้วย `bash test.sh` จะทำงานใน Subshell เมื่อสคริปต์จบ Subshell จะปิดตัวลง โฟลเดอร์ปัจจุบันของ Shell ตัวหลักจะไม่เปลี่ยน)

11	จากข้อ 10 ถ้าเรียกใช้ shell script นี้ โดยใช้คำสั่ง `source test.sh` เมื่อโปรแกรมทำงานจบ โฟลเดอร์ปัจจุบันที่เราจะใช้งานอยู่คือโฟลเดอร์ test ที่เพิ่งสร้างขึ้น	**จริง**	True (คำสั่ง `source` หรือ `.` จะรันคำสั่งทั้งหมดบน Shell ปัจจุบันโดยตรง)

12	`int main(int argc, char *argv[]) {`
	`  // ...`
	`}`
	เมื่อเรียกใช้โปรแกรมด้วยคำสั่ง:
	`./app Ungink 1234 5678`
	ค่าของ `argc` คือ
		D 4	D. 4 (นับ `./app` , `Ungink` , `1234` , `5678`)

13	จากโปรแกรมในข้อ 12 ค่าของ `argv[1]` คือ	**H. "Ungink"** - I. "1234" - J. "5678"	H. "Ungink" (`argv[0]` คือ `"./app"` , `argv[1]` คือ `"Ungink"`)

14	`public static void main(String[] args) {`
	`    // ...`
	`}`
	เมื่อเรียกใช้โปรแกรมด้วยคำสั่ง:
	`java app Ungink 1234 5678`
	ค่าของ `args[1]` คือ
		I. "1234" - J. "5678"	I. "1234" (ใน Java `args[0]` คือ `"Ungink"`, `args[1]` คือ `"1234"`)

15	`wait(NULL);` หมายถึง parent process จะไม่หยุดรอการทำงานของ child process	**ไม่จริง**	False (`wait(NULL)` สั่งให้ Parent หยุดรอจนกว่าลูกคนใดคนหนึ่งจะจบการทำงาน)

16	`#include <stdio.h>`
	`#include <stdlib.h>`
	`#include <unistd.h>`
	`#include <sys/wait.h>`
	`#include <sys/types.h>`
	
	`int main() {`
	`    pid_t pid;`
	`    int i;`
	`    for (i = 0; i < 3; i++) {`
	`        pid = fork();`
	`        if (pid == 0) {`
	`            if (i == 0) {`
	`                execlp("echo", "echo",`
	`"good morning", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else if (i == 1) {`
	`                execlp("echo", "echo",`
	`"good afternoon", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else {`
	`                execlp("echo", "echo",`
	`"good night", NULL);`
	`                printf("hi\n");`
	`            }`
	`        }`
	`    }`
	`    while(wait(NULL) != -1);`
	`    printf("goodbye\n");`
	`    return 0;`
	`}`
	จากโปรแกรมภาษา C ดังรูป จะมีผลลัพธ์กี่บรรทัด (หมายเหตุ: คำสั่ง echo มีเวอร์ชันที่เป็น external program)
		D 4	D. 4 `execlp` รัน echo 3 รอบ (3 บรรทัด) และไม่มีลูกตัวไหนมาทำ `printf("hi\n")` เพราะ `exec` แทนที่โปรเซสเดิมไปแล้ว จากนั้นแม่พิมพ์ `goodbye\n` อีก 1 บรรทัด รวมเป็น 4 บรรทัด

17	`#include <stdio.h>`
	`#include <stdlib.h>`
	`#include <unistd.h>`
	`#include <sys/wait.h>`
	`#include <sys/types.h>`
	
	`int main() {`
	`    pid_t pid;`
	`    int i;`
	`    for (i = 0; i < 3; i++) {`
	`        pid = fork();`
	`        if (pid == 0) {`
	`            if (i == 0) {`
	`                execlp("cd", "cd", "test",`
	`NULL);`
	`                printf("hi\n");`
	`            }`
	`            else if (i == 1) {`
	`                execlp("echo", "echo",`
	`"good afternoon", NULL);`
	`                printf("hi\n");`
	`            }`
	`            else {`
	`                execlp("echo", "echo",`
	`"good night", NULL);`
	`                printf("hi\n");`
	`            }`
	`        }`
	`    }`
	`    while(wait(NULL) != -1);`
	`    printf("goodbye\n");`
	`    return 0;`
	`}`
	จากโปรแกรมภาษา C ดังรูป จะมีผลลัพธ์กี่บรรทัด (หมายเหตุ: คำสั่ง cd เป็น shell built in คำสั่ง echo มีเวอร์ชันที่เป็น external program)
		D 7	D. 7 เมื่อ i=0 ลูกคนที่ 1 เรียก `execlp("cd", ...)` ซึ่งล้มเหลว (เพราะ cd เป็น built-in) ทำให้ทำงานต่อแล้วพิมพ์ `hi\n` (1) และลูกคนนี้ไม่ได้ exit จึงวนลูปไป fork ลูกต่ออีก 2 ตัว (พิมพ์ good afternoon, good night) แล้วตัวมันเองพิมพ์ `goodbye\n` ด...

18	จากโปรแกรมภาษา C ในข้อ 17 คำว่า "goodbye" จะถูกพิมพ์ออกมากี่บรรทัด	**B** 2	B. 2 (พิมพ์จากโพรเซสแม่หลัก 1 ครั้ง และพิมพ์จากลูกตัวแรกที่รอดมาจากการ exec ล้มเหลวอีก 1 ครั้ง)

19	จากโปรแกรมภาษา C ในข้อ 16 ผลลัพธ์ที่เป็นไปได้คือ	**A.** - B. - C. - D.
		`goodbye`
		`good morning`
		`good afternoon`
		`good night`
			A (เพราะ `while(wait(NULL) != -1);` บังคับให้แม่รอจนลูกๆ ทุกตัวทำงานเสร็จหมดก่อน "goodbye" จึงต้องอยู่ท้ายสุด)

L6. Thread — ชุดที่ 5.1 · 12 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	จุดประสงค์ของการเขียนโปรแกรมแบบ multithread ก็คือต้องการให้มีส่วนของโปรแกรมหลาย ๆ ส่วนทำงานขนานกันไป	**จริง**	True
2	ข้อเสียของการสร้าง Thread คือมันใช้ทรัพยากรมากกว่าการสร้าง process ย่อย	**ไม่จริง**	False (Thread ใช้ทรัพยากรน้อยกว่าการสร้าง Process มาก จึงเรียกว่า Lightweight Process)
3	Thread แต่ละตัวจะใช้ program counter ร่วมกัน	**ไม่จริง**	False (Thread แต่ละตัวต้องมี Program Counter และ Register เซตของตัวเอง)
4	Thread แต่ละตัวจะใช้ stack ร่วมกัน	**ไม่จริง**	False (Thread แต่ละตัวต้องมี Stack ส่วนตัวสำหรับเก็บ Local variables และ Function call)
5	Thread แต่ละตัวจะใช้ data section จากโพรเซสที่สร้างพวกมันขึ้นมาร่วมกัน	**จริง**	True (แชร์ Code, Data และ Files ร่วมกัน)
6	Thread ที่ถูกสร้างจาก process เดียวกันไม่จำเป็นต้องมีสิทธิเท่ากัน	**ไม่จริง**	False (Thread ทุกตัวในโปรเซสเดียวกันถือครองสิทธิการเข้าถึงของโปรเซสนั้นเท่ากันทั้งหมด)
7	JVM เป็นระบบปฏิบัติการแบบใด	**B** Single-process multi-threaded	B. Single-process multi-threaded
8	Windows 10 เป็นระบบปฏิบัติการแบบใด	**D** multi-process multi-threaded	D. multi-process multi-threaded
9	การจับคู่ระหว่าง user-thread และ kernel-thread แบบ many-to-one จะมีปัญหาด้านการทำงานพร้อมกันของ user thread ที่ถูกสร้างขึ้น	**จริง**	True (ถ้าเธรดใดเธรดหนึ่งเรียกบล็อกกิ้งซิสเต็มคอลล์ เธรดอื่นทั้งหมดในโปรเซสนั้นจะถูกบล็อกไปด้วย)
10	Thread แต่ละ Thread ที่ถูกสร้างขึ้นไม่จำเป็นต้องมีที่เก็บข้อมูลของตัวเอง เพราะมันจะใช้ข้อมูลจาก PCB	**ไม่จริง**	False (แต่ละเธรดต้องมี Thread Control Block: TCB สำหรับเก็บค่า State, Registers, Stack pointer)
11	การจับคู่ user-level กับ kernel-level threads ที่ใช้ทรัพยากรมากที่สุดคือ	**C** One-to-One	C. One-to-One (สร้าง 1 user thread ต้องสร้าง 1 kernel thread มารองรับ ทำให้เกิด overhead สูงสุด)
12	การจับคู่ user-level กับ kernel-level threads ที่ user-thread จะไม่ถูกบล็อกจากการทำงานของ user-thread อื่นอย่างแน่นอนคือ	**C** One-to-One	C. One-to-One

L7. Thread Programming Concepts — ชุดที่ 5.2 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ในการเขียนโปรแกรมภาษา C เพื่อใช้ library Pthread ชื่อเมท็อดที่จะให้ thread ไปทำงานต้องชื่อว่า runner	ไม่จริง	False (ตั้งชื่อฟังก์ชันเป็นชื่ออะไรก็ได้ ขอเพียง signature ตรงตามที่กำหนดคือ `void func(void param)`)
2	ฟังก์ชันของ Pthread library ที่ทำหน้าที่คล้าย ๆ ฟังก์ชัน fork ของการสร้าง process คือ	**B** pthread_create()	B. pthread_create()
3	ฟังก์ชันของ Pthread library ที่ทำหน้าที่คล้าย ๆ ฟังก์ชัน wait ของการสร้าง process คือ	**C** pthread_join()	C. pthread_join()
4	การเขียนประโยคต่อไปนี้ `pthread_exit(5);` คือการที่ thread ลูก ส่งค่า 5 กลับไปให้ thread แม่	ไม่จริง	False (พารามิเตอร์ของ `pthread_exit` รับพอยน์เตอร์ `void *retval` การส่งค่า integer 5 ดิบๆ โดยไม่ cast pointer ทำให้ผิดไวยากรณ์)
5	ถ้าเขียนโปรแกรมภาษา C ชื่อ test.c ที่มีการเรียกใช้ Pthread library ในการคอมไพล์และลิงก์เพื่อให้ได้โปรแกรมชื่อ test จะใช้คำสั่งใดต่อไปนี้	**D** gcc -o test test.c -lpthread	D. gcc -o test test.c -lpthread
6	การที่จะทำให้คลาสที่สร้างขึ้นมีคุณสมบัติของ thread นักเขียนโปรแกรมภาษา Java จะสร้างคลาสโดยสืบทอดคุณสมบัติมาจากคลาส	**B** Thread	B. Thread (`extends Thread`)
7	การที่จะทำให้คลาสที่สร้างขึ้นมีคุณสมบัติของ thread นักเขียนโปรแกรมภาษา Java จะอิมพลีเมนต์ interface ชื่อ	**A** Runnable	A. Runnable (`implements Runnable`)
8	ส่วนของโปรแกรมที่เปรียบเสมือน main โปรแกรมสำหรับ thread ที่สร้างขึ้นในภาษา Java จะถูกเขียนไว้ใน method ชื่ออะไร	**A** run	A. run (`public void run()`)
9	เมท็อดสำหรับ Java thread ที่ทำหน้าที่คล้าย ๆ wait() ของการสร้าง process คือ	**D** join	D. join
10	ใน Java การสร้าง object ของคลาสที่สืบทอดคุณสมบัติของคลาส Thread ขึ้นมา เท่ากับว่า thread เริ่มต้นทำงานแล้ว	ไม่จริง	False (ต้องเรียก `.start()` ก่อน เธรดจึงจะเริ่มทำงาน)
11	สมมติ Foo เป็นคลาสที่อิมพลีเมนต์ runnable interface และเราสร้าง object ของคลาส Foo ชื่อ obj ถ้าต้องการให้ thread อยู่ในสถานะ runnable เราจะใช้คำสั่งต่อไปนี้ `obj.start();`	ไม่จริง	False (Interface `Runnable` ไม่มีเมท็อด `start()` ต้องส่งไปให้คลาส Thread เช่น `new Thread(obj).start();`)
12	เมื่อเรียกเมท็อด start() ของ object ที่เป็น object ของคลาส Thread ในภาษา Java, thread จะอยู่ในสถานะ running	ไม่จริง	False (จะเข้าสู่สถานะ Runnable เพื่อรอ Scheduler เลือกไปรัน)
13	การที่ thread ที่ได้รับคำสั่งให้จบการทำงาน แล้วจบการทำงานเลย เราจะเรียกการอิมพลีเมนต์ thread ในแบบนี้ว่า	**A** Asynchronous Cancellation	A. Asynchronous Cancellation
14	Java thread รองรับการสั่งจบการทำงานของ thread ในรูปแบบที่เรียกว่า (ให้ตอบรูปแบบที่ใกล้เคียงที่สุด)	**D** Cooperative Mechanism	D. Cooperative Mechanism (การใช้ Flag หรือ `Thread.interrupt()` ตรวจสอบสถานะร่วมกัน)
15	Pthread รองรับเฉพาะ deferred cancellation	ไม่จริง	False (Pthread รองรับทั้ง Asynchronous และ Deferred Cancellation)
16	คำสั่ง sleep() จัดเป็น cancellation point สำหรับ Pthread	จริง	True
17	คำสั่ง printf() จัดเป็น cancellation point สำหรับ Pthread	ไม่จริง	False
18	แนวคิดการสร้าง Thread ไว้ล่วงหน้า เพื่อที่เมื่อต้องการใช้จะสามารถนำมาใช้ได้เลย โดยไม่ต้องเสียเวลาไปสร้างขึ้นมาใหม่ แนวคิดนี้มีชื่อเรียกว่า	**B** Thread Pool	B. Thread Pool
19	ถ้าต้องการสร้าง Thread pool ใน Java แบบไม่จำกัดขนาดไว้ล่วงหน้า เราจะใช้เมท็อดใดจากคลาส Executors	**A** newCachedThreadPool()	A. newCachedThreadPool()
20	จำนวน thread ใน Java thread pool ที่สร้างได้สูงสุดคือ 10 thread	ไม่จริง	False (สามารถกำหนดขนาดได้ตามความต้องการ หรือยืดหยุ่นได้ไม่จำกัดใน Cached Thread Pool)

21	```
	#include <stdio.h>
	#include <stdlib.h>
	#include <pthread.h>
	
	int sum = 0;
	
	void *runner(void *param) {
	    int i, upper = atoi(param);
	    sum = 0;
	    for (i = 1; i <= upper; i++)
	        sum += i;
	    printf("I am child thread, I am
	calculating\n");
	    pthread_exit(0);
	}
	
	int main(int argc, char *argv[]) {
	    pthread_t tid;
	    pthread_attr_t attr;
	    pthread_attr_init(&attr);
	    pthread_create(&tid, &attr, runner, argv[1]);
	    printf("I am mother thread, I will wait for my
	child thread\n");
	    pthread_join(tid, NULL);
	    printf("from my child sum = %d\n", sum);
	    return 0;
	}
	``
	จากโปรแกรมภาษา C ต่อไปนี้ ถ้าผู้ใช้ run โปรแกรมนี้โดยใช้คำสั่ง `./pthread 5` ผลลัพธ์ที่เป็นไปได้คือ
		C
		``
		I am child thread, I
		am calculating
		I am mother thread,
		I will wait for my
		child thread
		from my child sum =
		15
		``
			C (ผลรวม 1+2+3+4+5 = 15 และลำดับข้อความของ Child หรือ Mother จะขึ้นก่อนก็ได้ แต่ `sum = 15` ต้องอยู่หลัง `pthread_join()`)

22	```
	class SharedData {
	    private int sum;
	    public int getSum() {
	        return sum;
	    }
	    public void setCalculatesum(int num) {
	        sum = num * (num + 1) / 2;
	    }
	}
	
	class Child extends Thread {
	    private SharedData data;
	    int num;
	    public Child(SharedData data, int num) {
	        this.data = data;
	        this.num = num;
	    }
	    public void run() {
	        System.out.print("I am calculating ");
	        data.setCalculatesum(num);
	    }
	}
	
	public class Main {
	    public static void main(String[] args) {
	        int num = Integer.parseInt(args[0]);
	        SharedData data = new SharedData();
	        Child child = new Child(data, num);
	        child.start();
	        try {
	            child.join();
	        } catch (InterruptedException ie) {}
	        System.out.print(data.getSum() + " ");
	        System.out.println("Goodbye");
	    }
	}
	``
	จากโปรแกรมภาษา Java ต่อไปนี้ ถ้าผู้ใช้รันโปรแกรมโดยใช้คำสั่ง `java Main 6` ผลลัพธ์ที่เป็นไปได้คือ
		C I am calculating 21 Goodbye	C. I am calculating 21 Goodbye ((6 × 7)/(2) = 21)

L8. CPU Scheduling — ชุดที่ 6 · 31 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ส่วนของ OS ที่เรียกว่า Scheduler มีหน้าที่ในการทำ context switch	**ไม่จริง**	False (Dispatcher เป็นตัวทำ Context switch ส่วน Scheduler ทำหน้าที่เลือกโปรเซส)
2	ถ้า OS ไม่สามารถขัดจังหวะการทำงานของโพรเซสได้หลังจากที่ให้โพรเซสได้ใช้งาน CPU Scheduling algorithm ที่ใช้จะเป็นแบบ nonpreemptive	**จริง**	True
3	การเลือกโพรเซสเข้าใช้งาน CPU แบบ round robin จะให้ response time ที่ดี	**จริง**	True
4	waiting time คือเวลาที่โพรเซสรออยู่ใน waiting state	**ไม่จริง**	False (Waiting time หมายถึงเวลาที่โปรเซสต้องรออยู่ใน **Ready Queue**)
5	สำหรับ multi-level feed back queue โพรเซสจะเลื่อนจากคิวที่มีสถานะสูงกว่าลงสู่คิวที่มีสถานะต่ำกว่าเสมอ	**ไม่จริง**	False (สามารถเลื่อนขึ้นไปยังคิวที่มีลำดับสูงกว่าได้ เช่น กลไก Aging)
6	aging ช่วยแก้ปัญหาการอดตาย (starvation) ของโพรเซส	**จริง**	True (เพิ่ม Priority ให้โปรเซสที่รอนานขึ้นเรื่อยๆ)
7	ถ้า waiting time มากจะทำให้ turnaround time มีค่ามากตามไปด้วย	**จริง**	True (Turnaround Time = Burst Time + Waiting Time)
8	FCFS เป็น algorithm แบบ preemptive	**ไม่จริง**	False (FCFS เป็น Non-preemptive)
9	ถ้าการเลือกโพรเซสเข้าใช้งาน CPU ทำให้ response time ดีแล้ว จะทำให้ทุกโพรเซสมี turnaround time ดีขึ้นด้วย	**ไม่จริง**	False (เช่น Round Robin ให้ response time ดีมาก แต่บ่อยครั้ง turnaround time เฉลี่ยกลับแย่กว่า SJF)
10	SRTF เป็น nonpreemptive algorithm	**ไม่จริง**	False (SRTF คือ Preemptive SJF)

📊 **ข้อมูลสำหรับข้อ 11 - 16**

Process	Arrival Time	Burst Time
P1	0	8
P2	2	4
P3	1	1

อัลกอริทึม: Non-preemptive SJF

Gantt Chart:

- t=0: P1 เริ่มรันยาวจนจบที่ t=8 (เพราะเป็น Non-preemptive)
- t=8: ในคิวมี P2 (burst 4) และ P3 (burst 1) → P3 สั้นกว่า จึงรัน t=8 ถึง t=9
- t=9: P2 รันต่อจนจบที่ t=13

สรุปค่าเวลา:

- P1: เสร็จ 8, Turnaround = 8-0 = 8, Waiting = 8-8 = 0
- P2: เสร็จ 13, Turnaround = 13-2 = 11, Waiting = 11-4 = 7
- P3: เสร็จ 9, Turnaround = 9-1 = 8, Waiting = 8-1 = 7

11	turnaround time ของ P1 คือ	**8**	
12	turnaround time ของ P2 คือ	**11**	
13	turnaround time ของ P3 คือ	**8**	
14	waiting time ของ P1 คือ	**0**	
15	waiting time ของ P2 คือ	**7**	
16	waiting time ของ P3 คือ	**7**	

📊 **ข้อมูลสำหรับข้อ 17 - 22**

(ใช้ตารางเดียวกับข้อ 11 - 16) **อัลกอริทึม:** Preemptive SJF (SRTF)

Gantt Chart:

- t=0: P1 รัน (เหลือ 8)
- t=1: P3 เข้ามา (burst 1), P1 เหลือ 7 → P3 แย่ง CPU รัน t=1 ถึง t=2 (P3 จบ)
- t=2: P2 เข้ามา (burst 4), P1 เหลือ 7 → P2 สั้นกว่า จึงรัน t=2 ถึง t=6 (P2 จบ)
- t=6: P1 รันต่อที่เหลือ 7 หน่วย จนจบที่ t=13

สรุปค่าเวลา:

- P1: เสร็จ 13, Turnaround = 13-0 = 13, Waiting = 13-8 = 5
- P2: เสร็จ 6, Turnaround = 6-2 = 4, Waiting = 4-4 = 0
- P3: เสร็จ 2, Turnaround = 2-1 = 1, Waiting = 1-1 = 0

17	turnaround time ของ P1 คือ	**13**	
18	turnaround time ของ P2 คือ	**4**	
19	turnaround time ของ P3 คือ	**1**	
20	waiting time ของ P1 คือ	**5**	
21	waiting time ของ P2 คือ	**0**	
22	waiting time ของ P3 คือ	**0**	

📊 **ข้อมูลสำหรับข้อ 23 - 25**

Process	Arrival Time	Burst Time
P1	0	8
P2	2	5
P3	1	3

อัลกอริทึม: Round Robin (RR), Time Quantum = 2

Gantt Chart (q=2):

- t=0..2: P1 รัน (เหลือ 6) → ณ t=2 คิวคือ [P3, P2, P1]
- t=2..4: P3 รัน (เหลือ 1) → คิวคือ [P2, P1, P3]
- t=4..6: P2 รัน (เหลือ 3) → คิวคือ [P1, P3, P2]
- t=6..8: P1 รัน (เหลือ 4) → คิวคือ [P3, P2, P1]
- t=8..9: P3 รัน (เหลือ 0) → **P3 จบที่ t=9**
- t=9..11: P2 รัน (เหลือ 1) → คิวคือ [P1, P2]
- t=11..13: P1 รัน (เหลือ 2) → คิวคือ [P2, P1]
- t=13..14: P2 รัน (เหลือ 0) → **P2 จบที่ t=14**
- t=14..16: P1 รัน (เหลือ 0) → **P1 จบที่ t=16**

23	turnaround time ของ P1 คือ	**16**	16 - 0 = 16
24	turnaround time ของ P2 คือ	**12**	14 - 2 = 12
25	turnaround time ของ P3 คือ	**8**	9 - 1 = 8
26	<table><tr><th>Process</th><th>Arrival Time</th><th>Burst Time</th></tr><tr><td>P1</td><td>0</td><td>8</td></tr><tr><td>P2</td><td>2</td><td>4</td></tr><tr><td>P3</td><td>1</td><td>1</td></tr></table> turnaround time ของ P2 จะเป็นเท่าไร ถ้า Scheduling Algorithm ที่ใช้คือ RR ที่มี time quantum เป็น 4	**H. 7**	7 (P1 รัน t=0..4, จากนั้น P3 รัน t=4..5, P2 รัน t=5..9 ทำเสร็จที่ t=9; Turnaround = 9 - 2 = 7)
27	สำหรับ multilevel queue โพรเซสที่อยู่ใน ready queue ใด ready queue หนึ่งจะอยู่ใน queue นั้นโดยไม่เปลี่ยนไปอยู่ที่ ready queue อื่น	**จริง**	True (Multilevel Queue ธรรมดาจะย้ายคิวไม่ได้ ต่างจาก Multilevel Feedback Queue ที่ย้ายได้)
28	ถ้ากำหนด time quantum น้อย ก็มีโอกาสที่จะเกิด context switch มากขึ้น	**จริง**	True
29	ถ้าใช้ multilevel feedback queue ที่มี 3 คิว และแต่ละคิวเป็นดังนี้ - q0 ใช้ Round Robin (RR) มี time quantum 8 - q1 ใช้ Round Robin (RR) มี time quantum 16 - q2 ใช้ FCFS โดยทุก process จะถูกส่งเข้า q0 ก่อน โดยมีเงื่อนไขว่าแต่ละ process จะได...	**B** q1	B. q1 (ใน q0 ทำได้ 2 รอบ รอบละ 8 หน่วย = 16 หน่วย งานยังเหลืออีก 4 หน่วย จึงตกมาที่ q1 ซึ่งมี quantum 16 หน่วย จึงทำงานเสร็จสิ้นที่ q1)
30	ถ้ามี thread ที่มีสิทธิสูงกว่าถูกส่งเข้ามาในระบบ JVM อาจหยุดการทำงานของ thread ที่ทำงานอยู่ และให้ thread ดังกล่าวเข้าทำงานแทน	**จริง**	True (Preemptive Priority-based Scheduling)
31	ถ้าต้องการให้ Java thread เปิดทางให้ thread อื่นได้ทำงาน เมท็อดที่ต้องเรียกใช้คือ	**D** yield()	D. yield()

L9. Process Synchronization — ชุดที่ 9.1 · 18 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	การที่มีโพรเซสมากกว่าหนึ่งโพรเซสไปใช้แก้ข้อมูลที่ใช้งานร่วมกันโดยไม่มีการจัดจังหวะให้เหมาะสม ซึ่งอาจจะทำให้ข้อมูลผิดได้เรียกว่า	**C** Race Condition	C. Race Condition
2	ถ้าค่าที่อยู่ในตัวแปร banana มีค่าเป็น 7 และมีโพรเซสสองโพรเซสเข้ามาแก้ไขค่าตัวแปรตัวนี้ โดยโพรเซส A เพิ่มค่า banana ขึ้นไปอีก 2 และ process B ลดค่าของ banana ลง 1 ถ้าโพรเซส A และโพรเซส B ได้ทำงานโพรเซสละ 1 ครั้ง (ใครทำก่อนก็ได้) ถ้ามีการจัด...	**C** 8	C. 8 (7 + 2 - 1 = 8)
3	จากเงื่อนไขในข้อ 2 ถ้าไม่มีการจัดจังหวะอย่างเหมาะสม ค่าของตัวแปร banana จะมีค่าอะไรหลังจากโพรเซสทั้งสองจบการทำงาน	**F** 6 หรือ 8 หรือ 9	F. 6 หรือ 8 หรือ 9 (เกิด Race condition ขึ้นอยู่กับว่าตัวใดเขียนทับตัวใด หรือผลลัพธ์ปกติ)
4	แนวคิดหลักของการจัดจังหวะระหว่างโพรเซสในการเข้าใช้ทรัพยากรที่สามารถใช้งานร่วมกันได้คือ การล็อกทรัพยากรไม่ให้โพรเซสอื่นใช้ในขณะที่ตัวโพรเซสกำลังใช้งานทรัพยากรอยู่	**จริง**	True
5	ส่วนของโปรแกรมที่โพรเซสที่ถือครองทรัพยากรต้องทำงานให้เสร็จสิ้นก่อนที่จะปล่อยทรัพยากรให้โพรเซสอื่นใช้เรียกว่า	**D** Critical Section	D. Critical Section
6	ถ้าจะคิดขั้นตอนวิธีเพื่อแก้ไขเรื่อง Synchronization เพื่อให้สมบูรณ์จะต้องผ่านเงื่อนไขสามข้อ แต่ถ้าผ่านสามข้อไม่ได้ ข้อที่ต้องทำให้ผ่านให้ได้คือ	**C** Mutual Exclusion	C. Mutual Exclusion (เป็นกฎเหล็กสำคัญที่สุดว่าห้ามเข้าใช้งานพร้อมกันเด็ดขาด)
7	เงื่อนไขที่ต้องรับประกันว่า การเลือกโพรเซสจะเลือกจากโพรเซสที่ต้องการเข้าใช้งานเท่านั้นคือ	**B** Progress	B. Progress
8	เงื่อนไขที่ต้องรับประกันว่า ในขณะเวลาหนึ่งจะมีเพียง process เดียว ที่ได้ทำงานใน critical section คือ	**C** Mutual Exclusion	C. Mutual Exclusion
9	เงื่อนไขที่ต้องรับประกันว่า โพรเซสที่รอการใช้งานอยู่จะไม่รอไปเรื่อย ๆ โดยไม่มีจุดสิ้นสุดคือ	**A** Bounded Waiting	A. Bounded Waiting
10	ถ้าไม่ผ่าน progress แล้ว ก็จะไม่ผ่าน Bounded Waiting	**จริง**	True
11	กลไกการแก้ปัญหา Synchronization คือเมื่อ process ทำงานอยู่ในส่วนของ critical section แล้วมันจะไม่ถูกขัดจังหวะ	**ไม่จริง**	False (โพรเซสยังคงถูกสลับ Context หรือขัดจังหวะได้ เพียงแต่โพรเซสอื่นจะไม่สามารถเข้าสู่ Critical Section ของตนได้)
12	Bakery Algorithm ยอมให้มี process มากกว่าหนึ่ง process ที่มีหมายเลขคิวเดียวกันได้	**จริง**	True (กรณีหมายเลขคิวซ้ำกัน จะตัดสินด้วย Process ID ที่น้อยกว่า)
13	ลำดับหมายเลขคิวใน Bakery Algorithm รองรับ bounded waiting	**จริง**	True (เป็นระบบ First-Come, First-Served ตามเบอร์คิว)
14	จากขั้นตอนวิธีปัญหา bounded buffer ที่เป็นการพยายามจัดจังหวะของนักเขียนโปรแกรมเองตามที่เราเรียนกัน ตัวแปรที่เป็นตัวแปรที่ใช้ร่วมกันระหว่าง producer และ consumer คือ	**E** count และ buffer	E. count และ buffer (`in` เป็นของ Producer, `out` เป็นของ Consumer)
15	ตัวแปรที่ producer ใช้ในการอ้างถึงช่อง ใน buffer ที่มันจะเขียนข้อมูลคือ	**A** in	A. in
16	ในการแก้ปัญหา Bounded buffer แบบที่จัดจังหวะของนักเขียนโปรแกรมเองตามที่เราเรียนกัน เงื่อนไขหนึ่งในการทำงานให้สำเร็จคือ ฝั่ง consumer จะต้องอ่านข้อมูลให้เสร็จจนจบกระบวนการถึงการลดตัวนับจำนวนข้อมูลใน buffer แต่ producer สามารถถูกขัดจังหวะได้โ...	**ไม่จริง**	False (ทั้งคู่ต้องทำงานจนครบกระบวนการและไม่ถูกขัดจังหวะในจุดวิกฤต มิเช่นนั้นค่า `count` จะเพี้ยน)
17	ข้อจำกัดของ Peterson's algorithm คือมันรองรับแค่ Mutual Exclusion และ Progress แต่ไม่รองรับ Bounded Waiting	**ไม่จริง**	False (Peterson's algorithm ผ่านครบทั้ง 3 ข้อ รวมถึง Bounded Waiting)
18	ข้อจำกัดของ Bakery algorithm คือมันรองรับแค่ Mutual Exclusion และ Progress แต่ไม่รองรับ Bounded Waiting	**ไม่จริง**	False (Bakery algorithm ผ่านครบทั้ง 3 ข้อสำหรับกรณี N โพรเซส)

L10. Semaphore (Concept & Pthread) — ชุดที่ 9.2 · 25 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	การจัดจังหวะที่ให้โพรเซสติดลูปรอจนกว่าเงื่อนไขเป็นเท็จจึงจะเข้าทำงานได้เรียกว่าการจัดจังหวะที่มี busy waiting	**จริง**	True (หรือเรียกว่า Spinlock)
2	Semaphore เป็นกลไกในการจัดจังหวะที่ไม่มี busy waiting	**จริง**	True (ในการอิมพลีเมนต์แบบมาตรฐานคลาสสิกของ OS โพรเซสที่รอจะถูกจับเข้า Sleep/Wait queue โดยไม่เปลือง CPU time)
3	Semaphore ที่ใช้กับปัญหา critical section คือ counting semaphore	**ไม่จริง**	False (ใช้ Binary Semaphore หรือ Mutex Lock)
4	Binary semaphore มีชื่อเรียกอีกอย่างว่า	**B** Mutex lock	B. Mutex lock

5	`Semaphore S;`
	`wait(S)`
	`Critical Section`
	`signal(S)`
	เพื่อที่จะใช้ semaphore ในการแก้ปัญหา critical section ต้องกำหนดค่าเริ่มต้นให้ S เป็น
		B 1	B. 1

6	เมื่อใช้ semaphore แบบ ไม่มี busy waiting โพรเซสซึ่งรอการเข้าใช้ critical section จะรออยู่ใน wait state	**จริง**	True (Waiting Queue)
7	การใช้ Semaphore รับประกันว่าจะผ่านเงื่อนไขสามข้อของการแก้ปัญหา Critical Section	**ไม่จริง**	False (การใช้ Semaphore อย่างไม่ระมัดระวังอาจเกิด Deadlock หรือ Starvation ทำให้ไม่ผ่าน Bounded Waiting)
8	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่เหมือนคำสั่ง wait() ของ Semaphore ที่แสดงใน slide คือ	**C** sem_wait()	C. sem_wait()
9	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่เหมือนคำสั่ง signal() ของ Semaphore คือ	**B** sem_post()	B. sem_post()
10	ฟังก์ชันของ PThread Semaphore ที่ทำหน้าที่กำหนดค่าเริ่มต้นให้ semaphore คือ	**A** sem_init()	A. sem_init()

11	`#include <stdio.h>`
	`#include <pthread.h>`
	`#include <semaphore.h>`
	
	`int count = 0;`
	`sem_t mutex;`
	
	`void *thread_func(void *param) {`
	`  int i;`
	`  for (i = 0; i < 5; i++) {`
	`  sem_wait(&mutex);`
	`  count++;`
	`  sem_post(&mutex);`
	`  }`
	`  pthread_exit(0);`
	`}`
	
	`int main() {`
	`  pthread_t tid;`
	`  sem_init(&mutex, 0, 1);`
	`  pthread_create(&tid, NULL, thread_func, NULL);`
	`  pthread_join(tid, NULL);`
	`  printf("%d\n", count);`
	`  return 0;`
	`}`
	ผลลัพธ์ที่ได้จากโปรแกรมคือ
		C 5	C. 5 (วนลูปบวกค่า 5 ครั้ง มี semaphore ป้องกันอย่างถูกต้อง)

12	การแก้ปัญหา bounded buffer ในวิชานี้ใช้ตัวแปร semaphore กี่ตัว	**C** 3	C. 3 (ประกอบด้วย `mutex`, `empty`, `full`)
13	การแก้ปัญหา bounded buffer ในวิชานี้ใช้ counting semaphore กี่ตัว	**B** 2	B. 2 (คือ `empty` และ `full` ส่วน `mutex` เป็น binary semaphore)
14	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ producer ใช้เพื่อตรวจสอบว่า buffer ยังมีช่องว่างหรือไม่คือ	**A** empty	A. empty
15	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ consumer ใช้เพื่อตรวจสอบว่ามีข้อมูลใหม่ใน buffer หรือไม่คือ	**B** full	B. full
16	การแก้ปัญหา bounded buffer ในวิชานี้ semaphore ที่ใช้เพื่อให้แน่ใจว่า ณ.ขณะเวลาใดเวลาหนึ่งจะมีแค่ producer หรือ consumer เท่านั้นที่ได้อ่านหรือเขียน buffer คือ	**C** mutex	C. mutex
17	ปัญหา Dining philosopher ในวิชานี้ นักปราชญ์แต่ละคนจะเป็นตัวแทนของ	**A** process	A. process

18	ปัญหา Dining philosopher ในวิชานี้ ตะเกียบจะเป็นตัวแทนของ	**B** resource	B. resource (ทรัพยากรที่มีจำนวนจำกัด)
19	ปัญหา Dining philosopher ในวิชานี้ semaphore จะใช้เพื่อจัดสรรตะเกียบให้นักปราชญ์ เพื่อให้แน่ใจว่านักปราชญ์แต่ละคนจะได้กินข้าว	**ไม่จริง**	False (Semaphore ป้องกันแค่การแย่งตะเกียบอันเดียวกัน แต่ไม่ได้รับประกันว่าทุกคนจะได้กิน อาจเกิด Deadlock หรือ Starvation ได้)
20	การใช้ semaphore กับปัญหา dining philosopher โดยใช้ขั้นตอนวิธีในวิชานี้ อาจทำให้เกิดสิ่งที่เรียกว่า	**A** deadlock	A. deadlock (เมื่อนักปราชญ์ทุกคนหยิบตะเกียบข้างขวาพร้อมกัน)
21	การแก้ปัญหา readers-writers ในวิชานี้ใช้ semaphore กี่ตัว	**B** 2	B. 2 (คือ `mutex` สำหรับคุม `readcount` และ `wrt` สำหรับล็อกไฟล์ข้อมูล)
22	การแก้ปัญหา readers-writers ในวิชานี้ semaphore ที่ใช้จัดจังหวะระหว่าง readers และ writer คือ	**A** wrt	A. wrt
23	การแก้ปัญหา readers-writers ในวิชานี้ semaphore ที่ใช้เพื่อควบคุมการเพิ่มหรือลดจำนวน reader คือ	**B** mutex	B. mutex
24	การแก้ปัญหา reader-writers ในวิชานี้สามารถมี reader มากกว่าหนึ่งตัวอ่านข้อมูลได้พร้อม ๆ กัน	**จริง**	True (Shared Read)
25	การแก้ปัญหา reader-writers ในวิชานี้ reader คนแรกเป็นคนเริ่มการป้องกันไม่ให้ writer เข้ามาเขียนข้อมูลในขณะที่ยังมี readers อ่านข้อมูลอยู่ และเป็นคนที่ต้องปลดล็อกให้ writer เขียนข้อมูลได้ เมื่อไม่มี readers ที่ต้องการอ่านข้อมูลแล้ว	**ไม่จริง**	False (Reader คนแรกเป็นคนเริ่มล็อก `wrt` ก็จริง แต่คนที่เป็นคนปลดล็อกให้ Writer คือ **Reader คนสุดท้าย** ที่อ่านเสร็จ ไม่ใช่คนแรก) # รวมโจทย์ คำถาม และเฉลยวิชาระบบปฏิบัติการ (Operating Systems) ชุดที่ 10 - 14

L11. Monitor, Java Sync, Java Semaphore — ชุดที่ 10 · 34 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดของ monitor คือ การรวมเอา shared data และ ฟังก์ชันที่ต้องการทำงานกับ shared data มาไว้ร่วมกัน และยอมให้มีทีละโพรเซสได้ทำงานใน monitor	จริง	True (Monitor รับประกัน Mutual Exclusion โดยอนุญาตให้มีเพียง 1 Thread/Process ทำงานอยู่ภายใน ณ ขณะใดขณะหนึ่ง)
2	Monitor เป็นกลไกที่มีระดับต่ำกว่า semaphore	ไม่จริง	False (Monitor เป็นกลไกระดับสูง High-level synchronization construct ที่คอมไพเลอร์ช่วยจัดการความปลอดภัย ต่างจาก Semaphore ที่เป็น Low-level)
3	ประเภทของตัวแปรที่ monitor เตรียมไว้ให้เรียกใช้ เมื่อ process ที่ทำงานอยู่ใน monitor ต้องการจะรอเงื่อนไขบางอย่างคือตัวแปรที่เรียกว่า	**A** condition	A. condition (ตัวแปรประเภท `condition` เช่น `condition x, y;`)
4	ฟังก์ชันที่ process ที่ทำงานอยู่ใน monitor จะเรียกใช้เพื่อรอเงื่อนไขบางอย่างคือ	**D** wait	D. wait (เช่น คำสั่ง `x.wait()`)
5	process ที่รอการเข้าใช้ monitor จะรออยู่ในคิวที่เรียกว่า	**B** entry	B. entry (Entry Queue หรือ Entry Set)
6	ฟังก์ชันที่ process ที่ทำงานอยู่ใน monitor จะปลุก process ที่รอเงื่อนไขอยู่คือ	**C** signal	C. signal (เช่น `x.signal()`)
7	Process ที่รอเงื่อนไขที่จะเกิดขึ้น จะรออยู่ในคิวแยกต่างหากจากคิวสำหรับ process ที่รอการเข้าใช้ monitor	จริง	True (Process ที่รอเงื่อนไขจะรออยู่ใน Condition Queue ของตัวแปรเงื่อนไขนั้นๆ ซึ่งแยกจาก Entry Queue ของตัว Monitor)
8	ถ้าจะเทียบกับ process state ตัว process ที่รอการเข้าใช้ monitor เปรียบได้กับ process ที่อยู่ใน ready state	ไม่จริง	False (โพรเซสที่รอเข้ามอนิเตอร์ถือว่าถูกบล็อก ไม่สามารถเข้าทำงานได้จนกว่ามอนิเตอร์จะว่าง)
9	ถ้าจะเทียบกับ process state ตัว process ที่รอเงื่อนไขที่จะเกิดขึ้นหลังจากได้เข้าทำใน monitor แล้ว เปรียบได้กับ process ที่อยู่ใน ready state	ไม่จริง	False (เปรียบได้กับ Waiting/Blocked state เนื่องจากต้องรอสัญญาณ signal จากโพรเซสอื่น)
10	ขั้นตอนวิธีในการแก้ปัญหา Dining Philosophers โดยใช้ monitor ในวิชานี้ ถ้านักปราชญ์หิวข้าว เขาจะเริ่มจากหยิบตะเกียบด้านซ้ายมือมาถือไว้	ไม่จริง	False (ในวิธี Monitor จะตรวจสอบว่าตะเกียบทั้งสองข้างว่างพร้อมกันหรือไม่ก่อน ถ้าว่างจึงจะหยิบตะเกียบทั้งสองข้างพร้อมกัน)
11	ขั้นตอนวิธีที่ใช้ monitor ในวิชานี้มาแก้ปัญหา dining philosopher อาจเกิดปัญหา	**C** starvation	C. starvation (แม้แก้ปัญหา Deadlock ได้ แต่นักปราชญ์บางคนอาจถูกเพื่อนข้างเคียงสลับกันหยิบตะเกียบตัดหน้าตลอดเวลาจนอดตาย)
12	กลไกที่คล้าย ๆ monitor ที่ Java เตรียมไว้ให้ใช้คือ	**C** synchronization	C. synchronization (คีย์เวิร์ด `synchronized`)
13	กลไกการทำ synchronization ของ Java คือจะมีเมท็อดชื่อ wait() และ notify() ให้เรียกใช้ในการล็อกออบเจกต์	ไม่จริง	False (การล็อกออบเจกต์เกิดขึ้นโดยอัตโนมัติจากการเข้าสู่ `synchronized method` หรือ `synchronized block` ส่วน `wait()` และ `notify()` ใช้สำหรับปล่อยล็อกและรอ/ปลุกสัญญาณ)
14	เมื่อ thread เรียกใช้ เมท็อดที่เป็น synchronized method, thread นั้นจะเรียกใช้ synchronized method นั้นได้ก็ต่อเมื่อไม่มี thread อื่นเรียก synchronized method ใด ๆ ของ object นั้นอยู่ก่อน	จริง	True (การล็อกระดับ Instance Object ตัวเดียวกัน จะครอบคลุมทุก Synchronized Method ของ Object นั้น)
15	thread ที่เรียกใช้ synchronized method ไม่ได้ จะรออยู่ในคิวที่เรียกว่า wait set	ไม่จริง	False (จะรออยู่ใน Entry Set ส่วน Wait Set จะไว้สำหรับเธรดที่เรียกคำสั่ง `wait()`)
16	การปลดล็อกทรัพยากรของ object ที่ถูกล็อกโดยการเรียก synchronized method จะเกิดขึ้นเมื่อ นักเขียนโปรแกรมเรียกเมท็อด yield()	ไม่จริง	False (`yield()` ไม่ได้ปล่อยการถือครองล็อก จะปลดล็อกเมื่อทำงานออกจากเมท็อดหรือเรียก `wait()`)
17	ถ้ามีการเรียกฟังก์ชัน wait() ใน synchronized method ของ Java thread จะไปรออยู่ใน wait set	จริง	True
18	ถ้ามีการเรียกฟังก์ชัน notify() ใน synchronized method ของ Java ตัว thread จะย้ายจาก wait set ไปอยู่ใน entry set	จริง	True (เธรดที่ถูกปลุกจะย้ายไป Entry Set เพื่อรอแย่ง Lock กลับมาทำงานต่อ)
19	เมท็อด notify() จะใช้เลือก process ที่นักเขียนโปรแกรมระบุออกจาก wait set	ไม่จริง	False (ระบบ JVM จะสุ่มเลือกมาหนึ่งเธรด ไม่สามารถเจาะจงระบุเธรดได้)

20	```
	class SharedData {
	    private int value = 5;
	    public synchronized void increment() {
	        value++;
	    }
	    public synchronized void decrement() {
	        value--;
	    }
	    public int getValue() {
	        return value;
	    }
	}
	
	class Worker1 extends Thread {
	    SharedData data;
	    public Worker1(SharedData data) {
	        this.data = data;
	    }
	    public void run() {
	        for (int i = 0; i < 3; i++) {
	            data.increment();
	        }
	    }
	}
	
	public class Main {
	    public static void main(String[] args) {
	        SharedData data = new SharedData();
	        Worker1 w1 = new Worker1(data);
	        w1.start();
	        try {
	            w1.join();
	        } catch (InterruptedException e) {}
	        System.out.println(data.getValue());
	    }
	}
	``
	จากโปรแกรมภาษา Java ดังรูป ผลลัพธ์ของโปรแกรมคือ
		A 8	A. 8 (ค่าเริ่มต้น 5 + 3 = 8)

21	```
	class SharedData {
	    private int value = 5;
	    private boolean isAvailable = false;
	
	    public synchronized void increment() {
	        while (isAvailable) {
	            try { wait(); } catch
	(InterruptedException e) {}
	        }
	        System.out.println("increment");
	        isAvailable = true;
	        notify();
	    }
	
	    public synchronized void decrement() {
	        while (!isAvailable) {
	            try { wait(); } catch
	(InterruptedException e) {}
	        }
	        System.out.println("decrement");
	        isAvailable = false;
	        notify();
	    }
	
	    public int getValue() {
	        return value;
	    }
	}
	``
	จากโปรแกรมภาษา Java ดังรูป (มีการสลับรอบการทำงานด้วย `wait()` และ `notify()`) ผลลัพธ์ที่เป็นไปได้จากโปรแกรมคือ
		C
		``
		increment
		decrement
		increment
		decrement
		5
		``
			C (เพราะเงื่อนไข `wait()` และ `notify()` บังคับให้ `increment` และ `decrement` ทำงานสลับกันทีละครั้ง)

22	Method ของ Semaphore class ของ Java ที่ทำหน้าที่เหมือน wait() ของ Semaphore คือ	**A** acquire()	A. acquire()
23	Method ของ Semaphore class ของ Java ที่ทำหน้าที่เหมือน signal() ของ Semaphore คือ	**C** release()	C. release()

💻 **ข้อ 24 - 34:** โครงสร้างโค้ด BoundedBuffer ด้วย Semaphore ใน Java

```
import java.util.concurrent.Semaphore;

public class BoundedBuffer {
    private static final int BUFFER_SIZE = 5;
    private Object[] buffer = new Object[BUFFER_SIZE];
    private int in = 0, out = 0;

    // A, B, C: ประกาศและกำหนดค่าเริ่มต้นให้กับ Semaphores
    private Semaphore mutex = //A
    private Semaphore empty = //B
    private Semaphore full =  //C

    public void insert(Object item) {
        try {
            //D
            //E
            buffer[in] = item;
            in = (in + 1) % BUFFER_SIZE;
            //F
            //G
        } catch (InterruptedException e) {}
    }

    public Object remove() {
        Object item = null;
        try {
            //H
            //I
            item = buffer[out];
            out = (out + 1) % BUFFER_SIZE;
            //J
            //K
        } catch (InterruptedException e) {}
        return item;
    }
}
```

24	ที่จุด `//A` ต้องเติม	**C** `new Semaphore(1);`	Mutex Lock
25	ที่จุด `//B` ต้องเติม	**E** `new Semaphore(BUFFER_SIZE);`	นับจำนวนช่องว่าง
26	ที่จุด `//C` ต้องเติม	**D** `new Semaphore(0);`	เริ่มต้นยังไม่มีข้อมูล
27	ที่จุด `//D` ต้องเติม	**F** `empty.acquire();`	รอลดจำนวนช่องว่าง
28	ที่จุด `//E` ต้องเติม	**A** `mutex.acquire();`	เข้า Critical Section ใน insert
29	ที่จุด `//F` ต้องเติม	**B** `mutex.release();`	ออกจาก Critical Section ใน insert
30	ที่จุด `//G` ต้องเติม	**I.** **`full.release();`**	ส่งสัญญาณว่ามีข้อมูลพร้อมอ่านเพิ่มขึ้น 1
31	ที่จุด `//H` ต้องเติม	**H.** **`full.acquire();`**	รอลดจำนวนข้อมูลที่มีให้อ่าน
32	ที่จุด `//I` ต้องเติม	**A** `mutex.acquire();`	เข้า Critical Section ใน remove
33	ที่จุด `//J` ต้องเติม	**B** `mutex.release();`	ออกจาก Critical Section ใน remove
34	ที่จุด `//K` ต้องเติม	**G.** **`empty.release();`**	ส่งสัญญาณว่ามีช่องว่างเพิ่มขึ้น 1

L12. Deadlock — ชุดที่ 11 · 30 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	Deadlock จะเกิดขึ้นต้องมีตั้งแต่ 2 โพรเซสขึ้นไปเกี่ยวข้อง	**จริง**	True
2	เงื่อนไขที่จะทำให้เกิด deadlock ที่ระบุว่าทรัพยากรจะถูกใช้งานได้ทีละโพรเซสเท่านั้นคือ	**A** Mutual Exclusion	A. Mutual Exclusion
3	เงื่อนไขที่จะทำให้เกิด deadlock ที่ระบุว่าเมื่อทรัพยากรถูกจัดสรรให้โพรเซสแล้ว ทรัพยากรจะถูกนำมาให้โพรเซสอื่นใช้ได้ เมื่อโพรเซสที่ถือครองอยู่ยอมปล่อยทรัพยากรออกมาเท่านั้นคือ	**C** No Preemption	C. No Preemption
4	ถ้ามีเงื่อนไขข้อใดข้อหนึ่งใน 4 ข้อของ deadlock เป็นจริง จะทำให้เกิด deadlock	**ไม่จริง**	False (ต้องเป็นจริงครบทั้ง 4 ข้อพร้อมกัน จึงจะเกิด Deadlock)
5	สำหรับ Resource-Allocation Graph ที่ใช้ในการโมเดล deadlock จะเป็นกราฟแบบไม่มีทิศทาง	**ไม่จริง**	False (เป็น Directed Graph กราฟแบบมีทิศทาง)
6	Request edge ของ resource-allocation graph มีทิศทางจากทรัพยากรไปยังโพรเซส	**ไม่จริง**	False (Request Edge มีทิศทางจาก $P_i \rightarrow R_j$, ส่วน Assignment Edge มีทิศทางจาก $R_j \rightarrow P_i$)
7	ถ้ามี cycle ใน resource-allocation graph จะหมายความว่าเกิด deadlock ขึ้น	**ไม่จริง**	False (ถ้าทรัพยากรมีหลายชิ้น/multi-instance การมี cycle เป็นเพียงเงื่อนไขจำเป็น แต่อาจไม่เกิด deadlock ก็ได้)
	📊 **ข้อมูลสำหรับข้อ 8 - 14** (กราฟหน้า 2 และ 3) *โครงสร้างกราฟ:* • Processes: P_1, P_2, P_3 • Resources: R_1 (1 instance), R_2 (2 instances), R_3 (1 instance), R_4 (3 instances) • Edges: • $P_1 \rightarrow R_1$, $R_1 \rightarrow P_2$ • $R_2 \rightarrow P_1$, $R_2 \rightarrow P_2$, $P_3 \rightarrow R_2$ • $P_2 \rightarrow R_3$, $R_3 \rightarrow P_3$		
8	จากรูปมีทรัพยากรกี่ประเภท	**D** 4	R_1, R_2, R_3, R_4
9	จากรูปมีโพรเซส โพรเซส	**C** 3	P_1, P_2, P_3
10	จากรูปทรัพยากร R2 มีกี่ชิ้น	**B** 2	
11	จากรูปทรัพยากร R1 ถูกจัดสรรให้โพรเซสใด	**B** P2	
12	จากรูปโพรเซส P3 ถือครองทรัพยากรใด	**C** R3	มีลูกศรจาก $R_3 \rightarrow P_3$
13	จากรูปโพรเซส P3 ร้องขอทรัพยากรใด	**B** R2	ลูกศร $P_3 \rightarrow R_2$
14	จากรูประบบไม่เกิด deadlock	**ไม่จริง**	เกิด Deadlock เนื่องจากมี Cycle และทรัพยากรทุกตัวใน Cycle ถูกใช้งานจนเต็ม
15	OS ที่เราใช้กันทั่วไปไม่มีกลไกจัดการ deadlock ในระดับ user process	**จริง**	True (ส่วนใหญ่ใช้วิธี Ostrich Algorithm คือเพิกเฉยปล่อยให้เป็นหน้าที่ของผู้ใช้หรือโปรแกรมเมอร์)
16	กลไกการจัดการ deadlock ที่ไม่ยอมให้เกิด deadlock ขึ้นในระบบ	**A** Deadlock avoidance	A. Deadlock avoidance (รวมถึง Deadlock Prevention)
17	การกำหนดให้โพรเซสต้องขอใช้ทรัพยากรเรียงตามลำดับจากทรัพยากรที่มีหมายเลขน้อยไปหามาก จะใช้แก้ปัญหา no preemption	**ไม่จริง**	False (ใช้ป้องกันเงื่อนไข Circular Wait)
18	แนวคิดของ deadlock avoidance คือระบบต้องอยู่ใน safe state เสมอ	**จริง**	True
19	เงื่อนไขหนึ่งของ deadlock avoidance คือโพรเซสจะต้องระบุทรัพยากรที่ต้องการใช้ทั้งหมดก่อนที่มันจะเริ่มทำงาน	**จริง**	True (ต้องรู้ Max Claim ล่วงหน้า)
20	การใช้ resource-allocation graph ในการทำ deadlock avoidance จะใช้ได้กับกรณีที่ทรัพยากรหนึ่งประเภทมีเพียงหนึ่งชิ้นเท่านั้น	**จริง**	True (ถ้ามีหลายชิ้นต้องใช้อัลกอริทึม Banker's Algorithm)
21	ด้านที่เพิ่มเข้ามาใน resource allocation graph เพื่อใช้ในการทำ deadlock avoidance คือ	**B** Claim Edge	B. Claim Edge (เส้นประ $P_i \dashrightarrow R_j$)
22	ถ้าทดลองว่าให้ทรัพยากรแล้ว ระบบไม่อยู่ใน Safe State OS จะเปลี่ยน... เป็น...	**D** Request Edge, Claim Edge	D (หรือหากในขั้นตอนการทดสอบ ระบบจะปฏิเสธการขอและไม่เปลี่ยนเป็น Assignment edge)
23	ระบบจะอยู่ใน Safe State เมื่อ OS เปลี่ยน claim edge เป็น assignment edge แล้ว ไม่พบว่ามี cycle ใน graph	**จริง**	True
	📊 **ข้อมูลสำหรับข้อ 24 - 25** (กราฟหน้า 5) *โครงสร้าง:* P_1, P_2 และ R_1, R_2 โดยมี $R_1 \rightarrow P_1$, $P_2 \rightarrow R_1$, เส้นประ $P_1 \dashrightarrow R_2$, เส้นประ $P_2 \dashrightarrow R_2$		

24	จากรูป claim edge คือ	**A** P1 R2	
25	จากรูปถ้าโพรเซส P1 ร้องขอทรัพยากร R2 ระบบจะยอมให้การร้องขอนี้เกิดขึ้น	**ไม่จริง**	เพราะถ้าให้ R_2 แก่ P_1 จะเกิด Cycle และนำไปสู่ Unsafe state
26	กลไกการทำ detection เพื่อแก้ปัญหา critical section คือเปลี่ยน resource-allocation graph เป็น wait-for graph และตรวจหา cycle ใน wait-for graph	**ไม่จริง**	False (Wait-for graph ใช้สำหรับตรวจจับ **Deadlock** ไม่ใช่ Critical Section)
27	จากรูปหน้า 6 เมื่อเปลี่ยน resource-allocation graph เป็น wait-for graph แล้วจะพบว่า p2 ไม่ได้รอ p4 โดยตรง	**จริง**	True (ลูกศรผ่านทรัพยากรหลายทอด)
28	จากรูปหน้า 6 ทุกโพรเซสในระบบติดอยู่ใน deadlock	**ไม่จริง**	False (มีบางโพรเซส เช่น P_5 ที่สามารถทำงานจนเสร็จแล้วคืนทรัพยากรได้)
29	หลังจากที่มีการตรวจสอบพบว่าเกิด deadlock ขึ้นในระบบ วิธีเดียวในการแก้ปัญหาคือจบการทำงานของทุกโพรเซสที่เกี่ยวข้องกับ deadlock	**ไม่จริง**	False (สามารถใช้วิธียกเลิกทีละโพรเซส หรือทำ Resource Preemption ได้)
30	การแก้ปัญหา deadlock อาจทำให้เกิด starvation ได้	**จริง**	True (กรณีโพรเซสเดิมถูกเลือกเป็น Victim ให้ Rollback ซ้ำๆ)

L13. Real Memory Management (Contiguous scheme) — ชุดที่ 12.1 · 15 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ตำแหน่งของหน่วยความจำที่อ้างถึงโดย CPU คือ	**E** A หรือ C	E. A หรือ C (Logical address หรือ Virtual Address)
2	ตำแหน่งของหน่วยความจำที่อ้างถึงโดยฮาร์ดแวร์หน่วยความจำ	**B** Physical Address	B. Physical Address
3	ฮาร์ดแวร์ที่ใช้แปลงจาก logical address เป็น physical address	**A** Memory Management Unit	A. Memory Management Unit (MMU)
4	ไฟล์ที่มีนามสกุล .dll ของ Windows เป็นตัวอย่างของ	**B** Dynamic Linking	B. Dynamic Linking (Dynamic Link Library)
5	สำหรับแนวคิดของ dynamic linking ถ้าไลบรารีที่โพรเซสต้องการใช้ถูกโหลดเข้าหน่วยความจำโดยโพรเซสอื่นแล้ว โพรเซสนั้นจะสามารถใช้ไลบรารีตัวนั้นได้โดยไม่ต้องโหลดไลบรารีนั้นใหม่	**จริง**	True (Shared Library)
6	การจัดสรรหน่วยความจำให้กับโพรเซสแบบที่มีการแบ่งหน่วยความจำเป็นส่วน (partition) โดยที่ขนาดของแต่ละส่วนไม่จำเป็นต้องเท่ากัน แต่จะไม่สามารถเปลี่ยนจำนวนส่วนหรือขนาดของส่วนได้หลังจากเริ่มการทำงานของระบบไปแล้วเรียกว่า	**B** Fixed Partitions	B. Fixed Partitions
7	สำหรับการจัดสรรหน่วยความจำแบบ Fixed Partitions ณ. ขณะเวลาใดเวลาหนึ่งจะมีจำนวนโพรเซสสูงสุดได้เท่ากับจำนวนส่วน (partition) ที่แบ่งไว้	**จริง**	True (Multiprogramming Degree ถูกจำกัดด้วยจำนวนพาร์ติชัน)
8	สำหรับการจัดสรรหน่วยความจำแบบ Fixed Partitions การควบคุมไม่ให้โพรเซสไปใช้หน่วยความจำในส่วนอื่นนอกจากส่วนที่มันได้รับการจัดสรร จะใช้แนวคิดของ base และ limit รีจิสเตอร์ โดยที่ limit รีจิสเตอร์จะเก็บตำแหน่งเริ่มต้นของส่วนที่โพรเซสได้รับการจัดส...	**ไม่จริง**	False (สลับหน้าที่กัน: Base register เก็บตำแหน่งเริ่มต้น และ Limit register เก็บขนาด)
9	สมมติโพรเซสได้รับการจัดสรรลงในส่วนที่มีค่าใน base รีจิสเตอร์ 1000 และ limit รีจิสเตอร์ 2000 ถ้ามีการอ้าง logical address 1800 ถือว่าเป็นการอ้างตำแหน่งหน่วยความจำที่ถูกต้อง	**จริง**	True (1800 < 2000 จึงถูกต้อง)
10	สมมติโพรเซสได้รับการจัดสรรลงในส่วนที่มีค่าใน base รีจิสเตอร์ 1000 และ limit รีจิสเตอร์ 2000 ถ้ามีการอ้าง logical address 2000 ถือว่าเป็นการอ้างตำแหน่งหน่วยความจำที่ถูกต้อง	**ไม่จริง**	False (Logical Address ต้องน้อยกว่า Limit: 0 ≤ address < limit ดังนั้น 2000 จะเกิด Trap Addressing Error)
11	ปัญหาที่เกิดขึ้นกับวิธีการ Fixed Partitions ที่ทำให้มีหน่วยความจำเหลือว่างอยู่แต่ไม่มีโพรเซสใดสามารถใช้ได้เรียกว่า	**A** internal fragmentation	A. internal fragmentation
12	ปัญหาที่เกิดขึ้นกับวิธีการ variable Partitions ที่ทำให้มีหน่วยความจำเหลือว่างอยู่แต่ไม่มีโพรเซสใดสามารถใช้ได้เรียกว่า	**B** external fragmentation	B. external fragmentation
13	ในวิธี variable Partitions ถ้ามีเนื้อที่ว่างเหลืออยู่ในหน่วยความจำ เรียงตามตำแหน่งหน่วยความจำจากน้อยไปหามาก ดังต่อไปนี้ 16, 10, 5, 20 และ 3 MB ตามลำดับ ถ้ามี process ใหม่ที่ต้องการเนื้อที่หน่วยความจำ 4 MB และ OS จัดสรรให้ลงในช่อง 16 MB อัลก...	**E** first-fit	E. first-fit (เลือกช่องแรกสุดที่ขนาดเพียงพอ)
14	จากข้อมูลในข้อ 13 ถ้า OS จัดสรรให้ลงในช่อง 20 MB อัลกอริทึมที่ OS ใช้คือ	**C** worst-fit	C. worst-fit (เลือกช่องที่ใหญ่ที่สุด)
15	ในวิธี variable Partitions การขยับให้ process มาใช้เนื้อที่หน่วยความจำที่อยู่ติดกัน เพื่อให้เหลือเนื้อที่หน่วยความจำที่ว่างเรียงต่อกันเป็นผืนเดียวเรียกว่า	**G. compaction**	G. compaction

L14. Real Memory Management (Noncontiguous) — ชุดที่ 12.2 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดการจัดสรรหน่วยความจำแบบ paging คือหน่วยความจำ จะถูกแบ่งเป็นบล็อกขนาดเท่าๆ กัน แต่ละบล็อกจะเรียกว่า page	**ไม่จริง**	False (บล็อกขนาดเท่ากันใน Physical Memory จะเรียกว่า **Frame** ส่วน **Page** คือบล็อกของ Logical Memory)
2	การจัดสรรหน่วยความจำจริง (real memory) ในระบบ paging โปรแกรมจะสามารถถูกโหลดเข้าไปทำงานได้ ถ้าจำนวน frame ที่ว่างเท่ากับจำนวน page	**จริง**	True
3	ถ้า CPU อ้างถึงตำแหน่งหน่วยความจำโดยใช้จำนวนบิต 16 บิต ขนาดหน่วยความจำสูงสุดที่ระบบปฏิบัติการนี้จะใช้ได้คือ ไบต์	**E** 65536	E. 65536 ไบต์ (2^{16} = 65,536)
4	ถ้า CPU อ้างถึงตำแหน่งหน่วยความจำโดยใช้จำนวนบิต 16 บิต ถ้า 1 page มีขนาด 16 ไบต์ จะได้ว่าจาก 16 บิตดังกล่าวจะถูกแบ่งมาใช้ระบุ page offset จำนวนกี่บิต	**A** 4	A. 4 บิต (2^4 = 16)
5	จากข้อ 4 จะได้ว่าหนึ่งโปรแกรมจะมีจำนวน page ได้สูงสุดกี่ page	**D** 4096	D. 4096 (16 - 4 = 12 บิต สำหรับเลขหน้า, 2^{12} = 4096)
6	ในระบบ paging จะใช้เพื่อใช้แปลง logical address ให้เป็น physical address	**C** Page table	C. Page table
📊 **ข้อมูลสำหรับข้อ 7 - 10** (รูปหน้า 2 และ 3) *โครงสร้าง*: Logical memory ขนาด 16 bytes แบ่งเป็น 4 pages (เพจละ 4 bytes: 0-3, 4-7, 8-11, 12-15). Page table: Page 0 → Frame 5, Page 1 → Frame 6, Page 2 → Frame 1, Page 3 → Frame 2.			
7	จากรูปโปรแกรมถูกแบ่งออกเป็นกี่ page	**D** 4	
8	จากรูป page แรกของโปรแกรมถูกโหลดเข้า frame ใดในหน่วยความจำ	**E** 5	
9	จากรูปถ้าต้องการอ้างถึงตำแหน่งหน่วยความจำของตัว "g" (อยู่ที่ logical address 6) logical address จะอ้างถึง page ที่ และ offset ที่	**C** 1, 2	6 / 4 = 1, 6 mod 4 = 2
10	จากรูปตัว "g" ถูกเก็บไว้ที่ frame ที่ และ offset ที่	**E** 6, 2	Frame 6, Offset 2
11	ถ้าการเข้าถึง TLB ใช้เวลา 0.25 ไมโครวินาที และ การเข้าถึงหน่วยความจำใช้เวลา 1 ไมโครวินาที สมมติ Effective Access Time (EAT) มีค่า 1.35 ไมโครวินาที จะได้ว่า Hit Ratio มีค่ากี่เปอร์เซนต์	**E** 90	E. 90% EAT = h(0.25 + 1.0) + (1-h)(0.25 + 2.0) = 1.35 ⟹ 1.25h + 2.25 - 2.25h = 1.35 ⟹ h = 0.90
12	ในระบบ paging การป้องกันไม่ให้โพรเซสไปอ้างถึง page ที่มันไม่มีสิทธิใช้จะทำได้โดยกำหนดสิ่งที่เรียกว่า ไว้ใน page table	**D** valid-invalid bit	D. valid-invalid bit
13	สมมติผู้ใช้คนหนึ่งสั่งรันโปรแกรม vi สมมติอีกว่าส่วนที่เป็นโค้ดของโปรแกรม vi แบ่งเป็น 5 page และมี page ที่เป็น data 2 page ถ้าใช้แนวคิดของการ shared page ถ้าผู้ใช้อีกคนหนึ่งสั่งรันโปรแกรม vi จะมีจำนวนเฟรมในหน่วยความจำที่ใช้ร่วมกันระหว่าง 2 ...	**E** 5	E. 5 เฟรม (แชร์เฉพาะหน้าส่วนที่เป็น Code 5 หน้า)
14	แนวคิดของการจัดสรรหน่วยความจำแบบ segmentation ก็คือการแบ่งโปรแกรมออกเป็นส่วน ๆ แต่ละส่วนไม่จำเป็นต้องมีขนาดเท่ากัน แต่เวลาโหลดแต่ละส่วนเข้าสู่หน่วยความจำ จะต้องโหลดให้อยู่ต่อเนื่องกันในหน่วยความจำ	**ไม่จริง**	False (แต่ละเซกเมนต์กระจายตัวอยู่คนละตำแหน่งในแรมได้ ไม่จำเป็นต้องต่อเนื่องกันทั้งโปรแกรม)
15	ในระบบ segmentation แต่ละโพรเซสจะมี segment table เพื่อใช้แปลง logical address เป็น physical address	**จริง**	True
16	ใน segment table ส่วนที่ใช้ระบุตำแหน่งเริ่มต้นของ segment ในหน่วยความจำคือ	**A** base	A. base
17	ใน segment table ส่วนที่ใช้ระบุขนาด segment ในหน่วยความจำคือ	**C** limit	C. limit
18	**ตาราง Segment Table:** • Segment 0: limit 1000, base 1400 • Segment 1: limit 400, base 6300 • Segment 2: limit 400, base 4300 • Segment 3: limit 1100, base 3200 • Segment 4: limit 1000, base 4700 ถ้า logical address อยู่ในรูป segment number:offset และระบุค่ามาดังนี้ 2:300 จะได้ physical address คือ	**C** 4600	C. 4600 (Segment 2: limit 400, offset 300 < 400 ถูกต้อง; Base 4300 + 300 = 4600)
19	จาก segment table ในข้อ 18 ถ้า logical address ระบุมาเป็น 1:400 จะได้ physical address คือ	**E** error	E. error (Segment 1 มี limit 400, offset 400 ไม่น้อยกว่า limit จึงเกิด Trap Addressing Error)
20	การจัดสรรหน่วยความจำแบบ segmentation ยังมีปัญหาของ external fragmentation อยู่	**จริง**	True (เนื่องจากแต่ละเซกเมนต์มีขนาดไม่เท่ากัน)
21	ใน OS 16 บิต ถ้า page size คือ 256 ไบต์ และ 1 ช่องของ page table ใช้ 2 ไบต์ ถ้าใช้ single level paging และขนาดของโปรแกรมคือ 512 ไบต์ ขนาดของ page table คือกี่ไบต์	**D** 512	D. 512 ไบต์ Virtual Address Space 16 บิต, Page size 256 B (2^8) ⟹ ต้องมี 2^{16-8} = 256 Entries. ตารางขนาด 256 × 2 B = 512 ไบต์

22	ใน OS 16 บิต ถ้า page size คือ 256 ไบต์ และ 1 ช่องของ page table ใช้ 2 ไบต์ ถ้าใช้ 2-level paging โดยจำนวนบิตของ outer page table และ inner page table คือ 4 บิต และขนาดของโปรแกรมคือ 512 ไบต์ ขนาดของ page table คือกี่ไบต์	**A** 64	A. 64 ไบต์ Outer table = $2^4 \times 2$ B = 32 B. ขนาดโปรแกรม 512 B ต้องการ Inner table เพียง 1 ตัว = $2^4 \times 2$ B = 32 B. รวม 32 + 32 = 64 ไบต์

L15. Virtual Memory — ชุดที่ 13 · 22 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	แนวคิดของ virtual memory จะทำให้ขนาดของ logical address ใหญ่กว่า physical address ได้	**จริง**	True
2	แนวคิดของ virtual memory โปรแกรมไม่จำเป็นต้องถูกโหลดเข้าหน่วยความจำทั้งโปรแกรม	**จริง**	True (โหลดเฉพาะส่วนที่กำลังใช้งาน)
3	ในการใช้ virtual memory แบบ demand paging เรายังใช้ page table ในการแปลง logical address เป็น physical address	**จริง**	True
4	บิตใน page table ที่ใช้เพื่อระบุว่า page ถูกโหลดเข้าสู่หน่วยความจำหรือยังคือ	**D** valid-invalid bit	D. valid-invalid bit
5	ในกรณีที่มีการอ้างถึง page ที่ไม่อยู่ในหน่วยความจำจะเกิดสิ่งที่เรียกว่า	**C** page fault	C. page fault
6	page fault จะเกิดขึ้นเมื่อโพรเซสอ้างถึงหน่วยความจำที่ตัวเองไม่มีสิทธิเข้าถึงเท่านั้น	**ไม่จริง**	False (เกิดขึ้นเมื่อโปรเซสอ้างถึงตำแหน่งที่มีสิทธิเข้าถึง แต่หน้านั้นยังไม่ได้ถูกดึงเข้ามาในแรม)
7	เมื่อเกิด page fault เนื่องจากส่วนของโปรแกรมที่ต้องการยังไม่ได้ถูกโหลดเข้าสู่หน่วยความจำ ระบบปฏิบัติการจะทำงานต่อไปนี้ยกเว้น	**E** เปลี่ยนค่า valid-invalid bit เป็น invalid	E. เปลี่ยนค่า valid-invalid bit เป็น invalid (ต้องเปลี่ยนเป็น **valid** เพราะโหลดเข้ามาสำเร็จแล้ว)
8	ในกรณีที่เกิด page fault และ ในหน่วยความจำไม่มีเฟรมที่ว่างเหลือแล้ว ระบบปฏิบัติการจะแจ้งข้อความข้อผิดพลาดให้ผู้ใช้ทราบ	**ไม่จริง**	False (OS จะเรียกกระบวนการ Page Replacement เพื่อสลับเพจเก่าออก)
9	วิธีวัดประสิทธิภาพของอัลกอริทึมการแทนที่เพจ ก็คือดูจากการเกิด page fault ถ้าอัลกอริทึมไหนเกิด page fault มากกว่า อัลกอริทึมนั้นจะดีกว่า	**ไม่จริง**	False (อัลกอริทึมที่ดีกว่าต้องเกิด Page Fault **น้อยกว่า**)
📊 ข้อมูลสำหรับข้อ 10 - 16 • หน่วยความจำมี 3 เฟรม ปัจจุบันเก็บเพจ: `2, 0, 1` (ลำดับการเข้า: 2 มาก่อน ตามด้วย 0 และ 1) • Page Reference String ที่เหลือ: `7 0 1 2 0 3 0 4 2 3 0`			
10	ด้วย FIFO เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	
11	ด้วย FIFO จะเกิด page fault ทั้งหมดกี่ครั้ง	**E** 7	
12	ด้วย Optimal เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	เพจ 2 ถูกใช้ไกลสุดในอนาคต
13	ด้วย Optimal จะเกิด page fault ทั้งหมดกี่ครั้ง	**C** 5	
14	วิธีการ optimal เป็นวิธีการที่ระบบปฏิบัติการนิยมใช้ เนื่องจากให้ผลลัพธ์เป็นจำนวน page fault น้อยที่สุด	**ไม่จริง**	เป็นวิธีในอุดมคติ ไม่สามารถนำมาใช้จริงได้เพราะไม่สามารถล่วงรู้อนาคต
15	ด้วย LRU เมื่อมีการเรียกใช้ page 7 จะแทนที่เพจใด	**C** 2	
16	ด้วย LRU จะเกิด page fault ทั้งหมดกี่ครั้ง	**E** 7	
17	การเลือกเฟรมที่จะถูกแทนที่จะถูกเลือกจากเฟรมที่ระบบปฏิบัติการจัดสรรให้กับโพรเซสนั้นเท่านั้น	**ไม่จริง**	False (มีทั้ง Local Replacement และ Global Replacement ที่ดึงเฟรมของโพรเซสอื่นได้)
18	เหตุการณ์ที่ระบบต้องสลับเพจเข้าและออกอยู่เกือบตลอดเวลาเรียกว่า	**D** thrashing	D. thrashing
19	ถ้าเกิด thrashing แล้วระบบปฏิบัติการตรวจสอบไม่พบ อาจจะเป็นผลให้ระบบปฏิบัติการโหลดโพรเซสเข้ามามากขึ้น เนื่องจาก CPU ไม่ถูกใช้งานเต็มที่	**จริง**	True (CPU Utilization ตกต่ำ ทำให้ OS นึกว่าระบบว่าง จึงเพิ่ม Multiprogramming Degree จนยิ่งแย่ลง)
20	สมมติ working set window คือ 10 และที่เวลา 11 โพรเซส p1 มี page reference string ดังนี้ `2 6 1 5 7 7 7 7 5 1` และ p2 มี page reference string เป็น `1 2 0 3 4 5 6 6 6 7` คอมพิวเตอร์จะต้องมีเฟรมว่างอย่างน้อยกี่เฟรม จึงจะไม่เกิด thrashing	**C** 13	C. 13 เฟรม ($WSS(P_1) = \{1, 2, 5, 6, 7\} = 5$; $WSS(P_2) = \{0, 1, 2, 3, 4, 5, 6, 7\} = 8$; รวม 5 + 8 = 13)

21	```
	public class Main {
	    public static void main(String[] args) {
	        int[][] a = new int[10][256];
	        for(int j = 0; j < 256; j++) {
	            for(int i = 0; i < 10; i++) {
	                a[i][j] = 0;
	            }
	        }
	    }
	}
	``
	สมมติ 1 แถวของอะเรย์ใช้เนื้อที่ 1 เพจ และหน่วยความจำมีเนื้อที่ว่าง 1 เฟรม ในการกำหนดค่าเริ่มต้นให้อะเรย์ a จะเกิด page fault กี่ครั้ง
		D 2560	D. 2560 ครั้ง (วนลูปตามหลัก Column-major เข้าถึงข้ามแถวทุกรอบ เกิด Page Fault ทุกครั้งที่เข้าถึง 256 × 10 = 2560)

22	```
	public class Main {
	    public static void main(String[] args) {
	        int[][] a = new int[10][256];
	        for(int i = 0; i < 10; i++) {
	            for(int j = 0; j < 256; j++) {
	                a[i][j] = 0;
	            }
	        }
	    }
	}
	``
	สมมติ 1 แถวของอะเรย์ใช้เนื้อที่ 1 เพจ และหน่วยความจำมีเนื้อที่ว่าง 1 เฟรม ในการกำหนดค่าเริ่มต้นให้อะเรย์ a จะเกิด page fault กี่ครั้ง
		A 10	A. 10 ครั้ง (วนลูปตามแถว Row-major เพจแถวเดิมถูกโหลด 1 ครั้งแล้วใช้งานต่อเนื่องครบ 256 ช่อง จึงเกิด Page Fault เพียงแถวละ 1 ครั้ง รวม 10 ครั้ง)

L16. File System — ชุดที่ 14 · 30 ข้อ

ข้อ	คำถาม	คำตอบ	เฉลย / ความรู้
1	ระบบไฟล์ทำให้การเข้าถึงไฟล์ผ่านทางซอฟต์แวร์มีลักษณะเดียวกันไม่ว่าอุปกรณ์ที่จัดเก็บข้อมูลจะเป็นอะไร	จริง	True (File System Abstraction)
2	หน้าที่หนึ่งของระบบไฟล์คือควบคุมสิทธิในการเข้าใช้งานไฟล์	จริง	True
3	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Windows เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**B** NTFS	B. NTFS
4	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Linux เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**C** ext4	C. ext4
5	ระบบไฟล์ที่ใช้กับระบบปฏิบัติการ Apple เป็นหลัก (เข้ากันไม่ค่อยได้กับระบบปฏิบัติการอื่น ๆ)	**D** APFS	D. APFS (Apple File System)
6	ระบบไฟล์ที่เข้ากันได้กับระบบปฏิบัติการหลาย ๆ ตัว	**A** exFAT	A. exFAT
7	ระบบไฟล์ที่มักจะใช้กับ External Storage อย่าง USB drive คือ	**A** exFAT	A. exFAT
8	ประโยชน์ของคุณสมบัติ Journaling ของระบบไฟล์บางตัวอย่าง NTFS คือ การเข้ารหัสไฟล์	ไม่จริง	False (Journaling ใช้บันทึก Log การเปลี่ยนแปลงโครงสร้างไฟล์เพื่อกู้คืนเมื่อระบบล่ม ไม่ใช่การเข้ารหัส)
9	ปัญหาของการจัดสรรเนื้อที่ให้กับไฟล์แบบต่อเนื่อง (contiguous) คือ internal fragmentation	ไม่จริง	False (ปัญหาสำคัญที่สุดของการจัดสรรต่อเนื่องคือ **External Fragmentation**)
10	ปัญหาของการจัดสรรเนื้อที่ให้กับไฟล์แบบต่อเนื่อง (contiguous) คือ การเพิ่มขนาดของไฟล์อาจจะทำได้ยาก	จริง	True (หากบล็อกถัดไปไม่ว่าง จะต้องย้ายไฟล์ทั้งไฟล์ไปที่อื่น)
11	การจัดสรรเนื้อที่ให้ไฟล์แบบ linked allocation เป็นการจัดสรรเนื้อที่ให้ไฟล์แบบต่อเนื่อง (contiguous)	ไม่จริง	False (Linked Allocation เป็นแบบ Non-contiguous)
12	ข้อเสียของการจัดสรรเนื้อที่แบบ linked allocation คือจะต้องเสียเนื้อที่ในบล็อกเพื่อเก็บลิงก์ไปยังบล็อกถัดไป	จริง	True (Pointer Overhead)
13	ข้อเสียของการจัดสรรเนื้อที่แบบ linked allocation คือถ้ามีบล็อกใดบล็อกหนึ่งเสีย อาจทำให้อ่านข้อมูลที่เหลือของไฟล์ไม่ได้	จริง	True (ลิงก์ขาดตอน)
14	การจัดสรรเนื้อที่แบบ linked allocation มีข้อเสียคือการอ่านไฟล์ทำได้ช้า	จริง	True (ไม่รองรับ Random/Direct Access ต้องไล่อ่านตามลำดับ)
15	FAT จัดเป็น Linked Allocation	จริง	True (เป็น Linked Allocation ที่ดึงพอยน์เตอร์มาเก็บรวมในตาราง FAT)
16	ขนาดสูงสุดของไฟล์สำหรับระบบที่ใช้ระบบไฟล์แบบ FAT32 คือ	**C** 4 GB	C. 4 GB (2^{32} - 1 bytes)
17	สมมติหนึ่งช่องของ FAT Table มีขนาด 16 บิต และทั้ง 16 บิตถูกใช้อ้างถึงบล็อกข้อมูล โดยขนาดของบล็อกข้อมูลหนึ่งบล็อกคือ 16K ขนาดของวอลุม (volume) สูงสุดของดิสก์คือกี่ GB	**A** 1	A. 1 GB (2^{16} = 65,536 บล็อก × 16 KB = 1,048,576 KB = 1 GB)
18	ปัญหาของขนาดของบล็อกที่มีขนาดใหญ่คือ external fragmentation	ไม่จริง	False (บล็อกใหญ่ทำให้เกิดปัญหา **Internal Fragmentation**)
19	ข้อดีประการหนึ่งของการเข้าถึงไฟล์แบบ index allocation คือ การเข้าถึงข้อมูลในไฟล์ไม่จำเป็นต้องเข้าถึงตามลำดับ	จริง	True (รองรับ Direct/Random Access)
20	ในการใช้งาน index allocation แบบ single level ขนาดของไฟล์สูงสุดจะถูกจำกัดด้วยขนาดของจำนวนช่อง (entry) ใน index block	จริง	True (เพราะมี Index block ได้เพียงก้อนเดียว)
21	สมมติ 1 บล็อกมีขนาด 2 KB และ pointer มีขนาด 4 bytes จะได้ว่า 1 index block จะมีจำนวนช่อง (entry) กี่ช่อง	**C** 512	C. 512 ช่อง (2048 / 4 = 512)
22	สมมติ 1 บล็อกมีขนาด 2 KB และ pointer มีขนาด 4 bytes ขนาดสูงสุดของไฟล์ที่เก็บได้ใน Double indirect block อย่างเดียวคือกี่ MB	**C** 512	C. 512 MB 512 × 512 = 262,144 data blocks × 2 KB = 524,288 KB = 512 MB
23	Symbolic link มีชื่อเรียกอีกอย่างว่า hard link	ไม่จริง	False (เรียกว่า Soft Link)
24	Symbolic link เปรียบได้กับการสร้าง shortcut ใน Windows	จริง	True
25	Hard Link ในระบบปฏิบัติการ Unix จัดการโดยใช้แนวคิด Reference Counter	จริง	True (นับ Link count ใน inode)
26	เราสามารถสร้าง Hard Link ไปยังไฟล์หรือไดเรกทอรีก็ได้	ไม่จริง	False (ระบบ Unix ส่วนใหญ่ไม่อนุญาตให้สร้าง Hard link ไปยังไดเรกทอรีเพื่อป้องกัน Loop ในโครงสร้างระบบไฟล์)
27	เราสามารถสร้าง Soft Link ไปยังไฟล์หรือไดเรกทอรีก็ได้	จริง	True
28	คำสั่ง softln ใน Unix ใช้สร้าง Soft Link	ไม่จริง	False (ใช้คำสั่ง `ln -s`)

29	เราไม่สามารถสร้าง mount point มากกว่าหนึ่ง mount point ที่ mount อุปกรณ์เดียวกันได้	**ไม่จริง**	False (สามารถ Mount อุปกรณ์เดียวกันไว้ที่หลายโฟลเดอร์/Mount point ได้)
30	ถ้าต้องการกำหนดสิทธิในการเข้าถึงไฟล์ดังนี้ - เจ้าของ: อ่านได้ เขียนได้ execute ได้ (`rwx` = 4+2+1 = 7) - ผู้ใช้ในกลุ่มเดียวกัน: อ่านได้ เขียนได้ execute ไม่ได้ (`rw-` = 4+2+0 = 6) - คนอื่นในระบบ: อ่านได้อย่างเดียว (`r--` = 4+0+0 = 4) สิทธิการเข้า...	**C** 0764	C. 0764